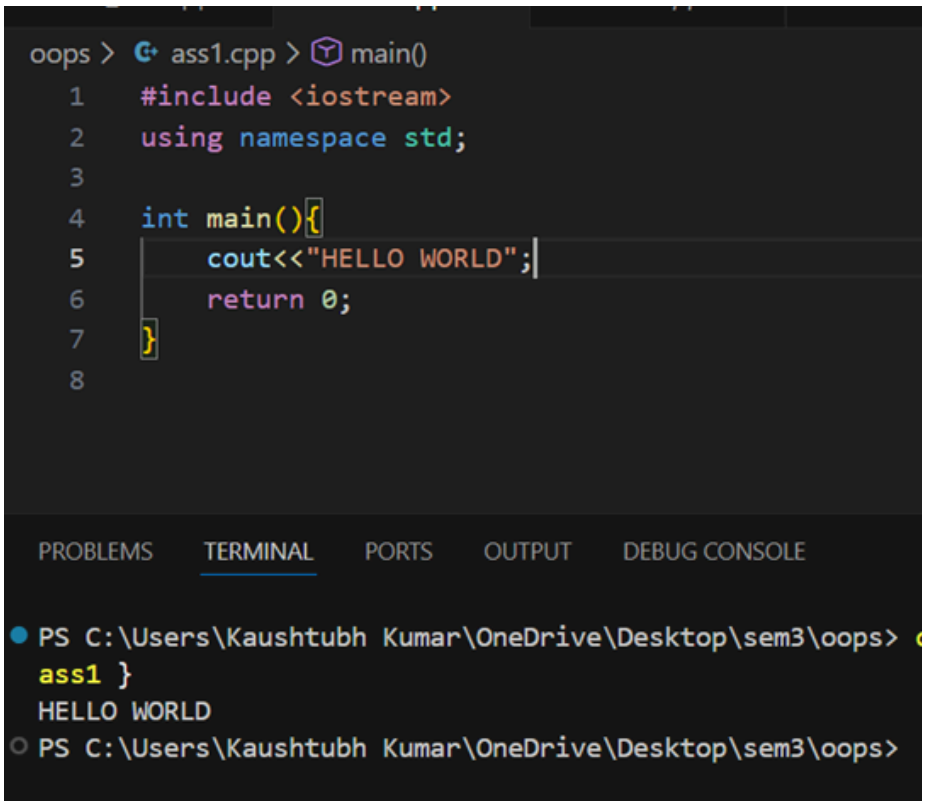
ASSIGNMENT-01

Q1 LAB -1

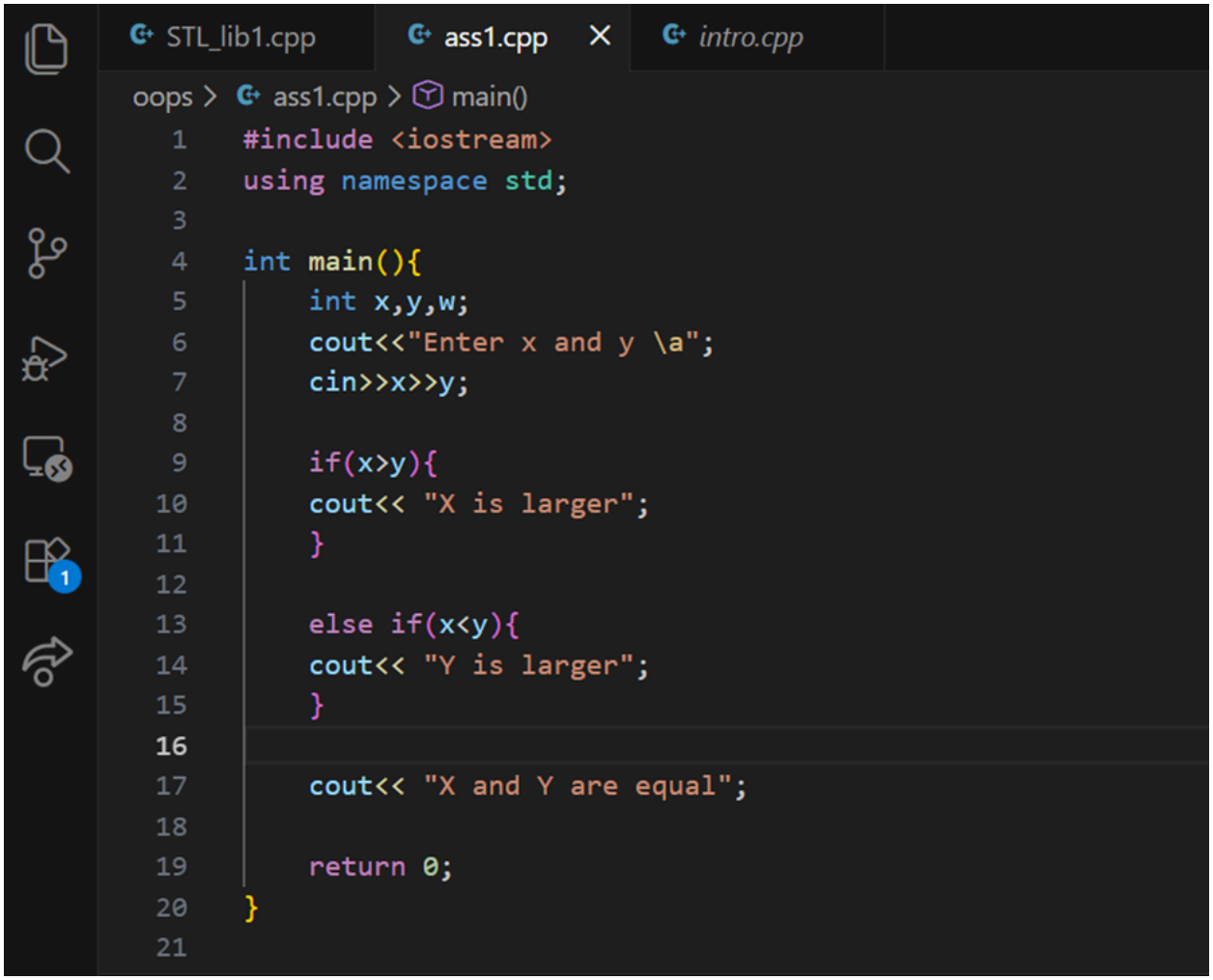
Topic Covered - cin/cout basics, structures in C, structures in C++, Classes and objects

Programming Question

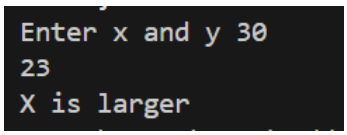
1. Write a program (WAP) to display "Hello World" on console display.

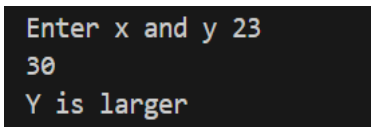


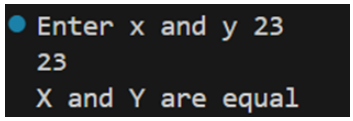
2. WAP to that reads two values from keyboard and displays the larger value on the screen.



OUTPUTS:



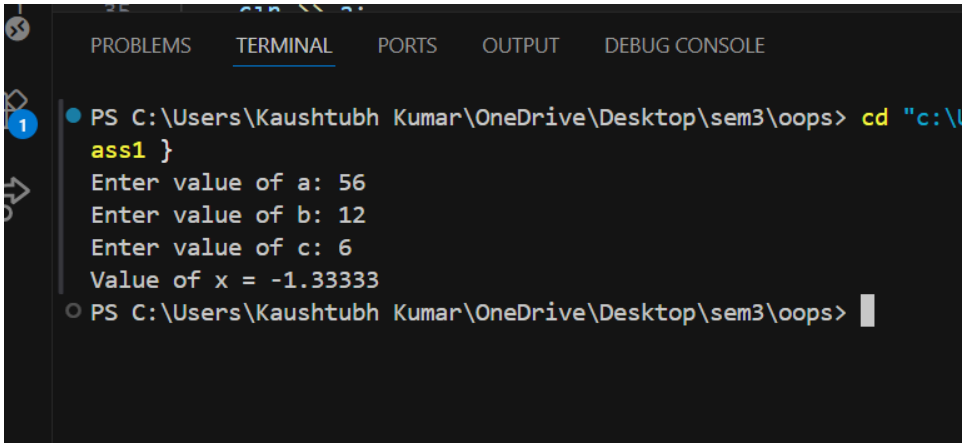




3. WAP to read three values a, b, c from keyboard and display the value of x = a/b-c.

```cpp
#include <iostream>
using namespace std;

int main() {
    float a, b, c, x;

    cout << "Enter value of a: ";
    cin >> a;
    cout << "Enter value of b: ";
    cin >> b;
    cout << "Enter value of c: ";
    cin >> c;

    x = a / b - c;

    cout << "Value of x = " << x << endl;

    return 0;
}
```

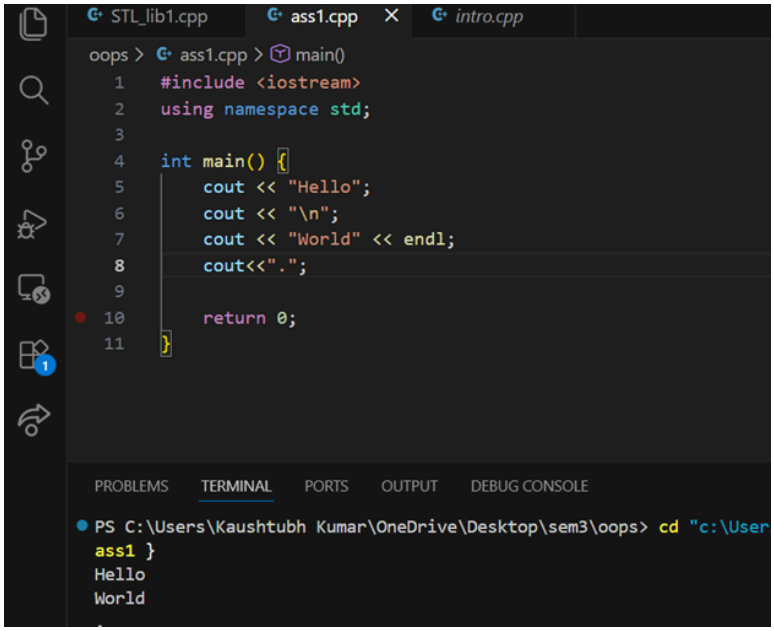
OUTPUT:



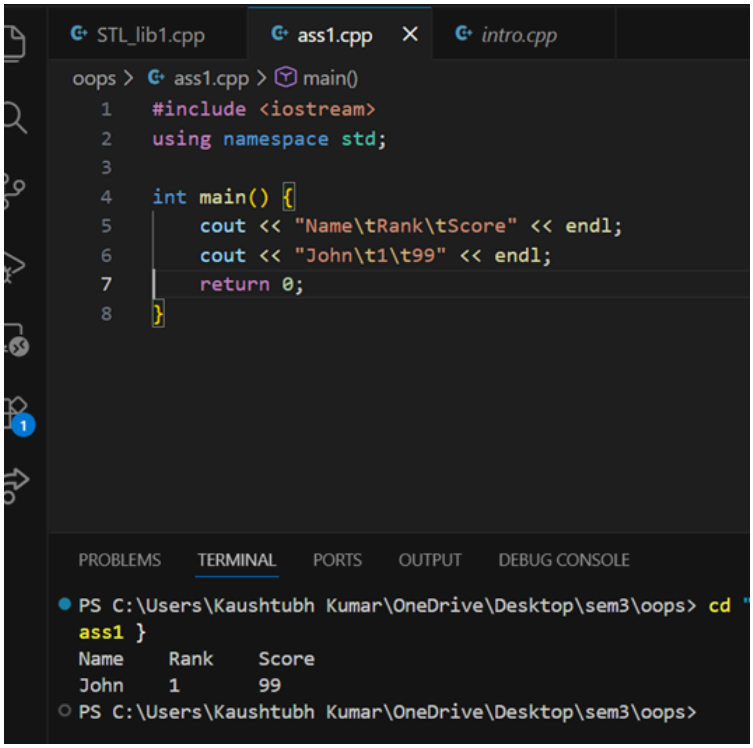
4. WAP to implement the following control characters:

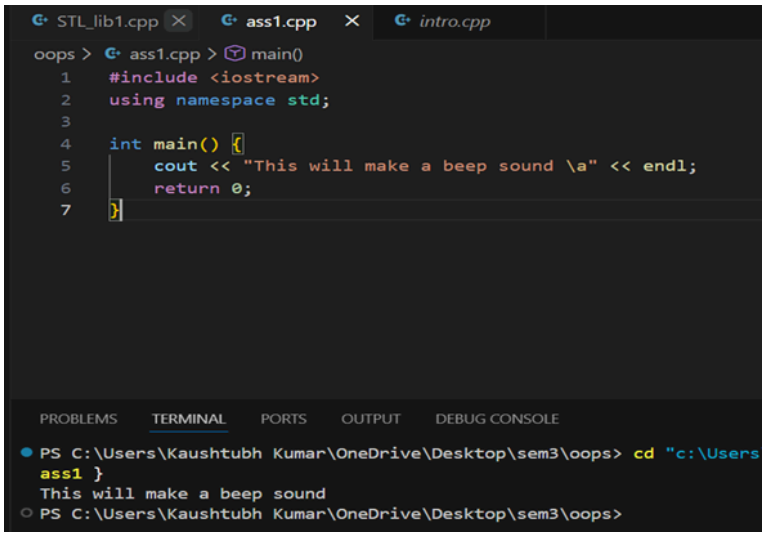
- *'\n' is for new line, or you can use endl –*

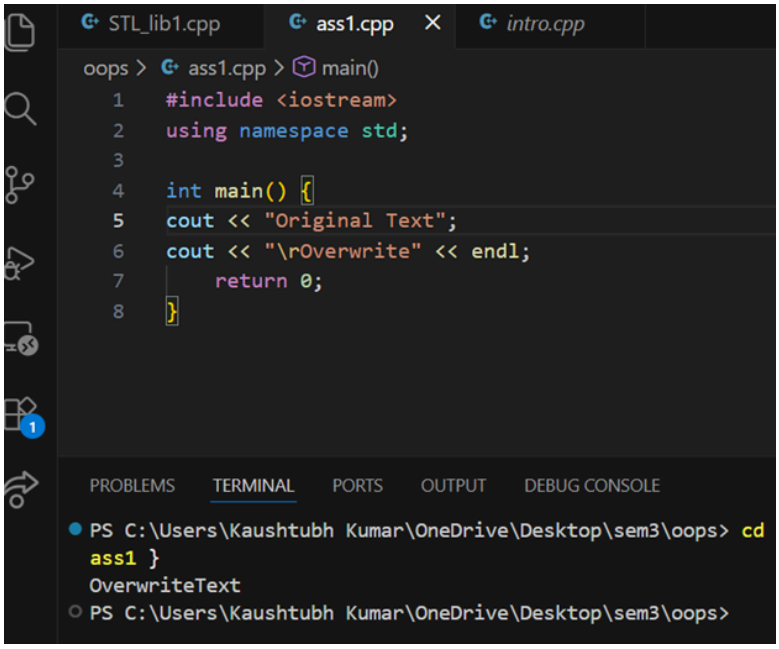
  cout<<endl<<"message";
- *'\t' is for tab*
- *'\a' is an alarm sound*
- *'\r' is carriage return to go to the beginning of the current line*



KAUSHTUBH KUMAR 1024150191 2O23 OOPS_LAB







5. Implement *namespace* in a program to illustrate the use of same name variables and functions in different sections/libraries of the code.

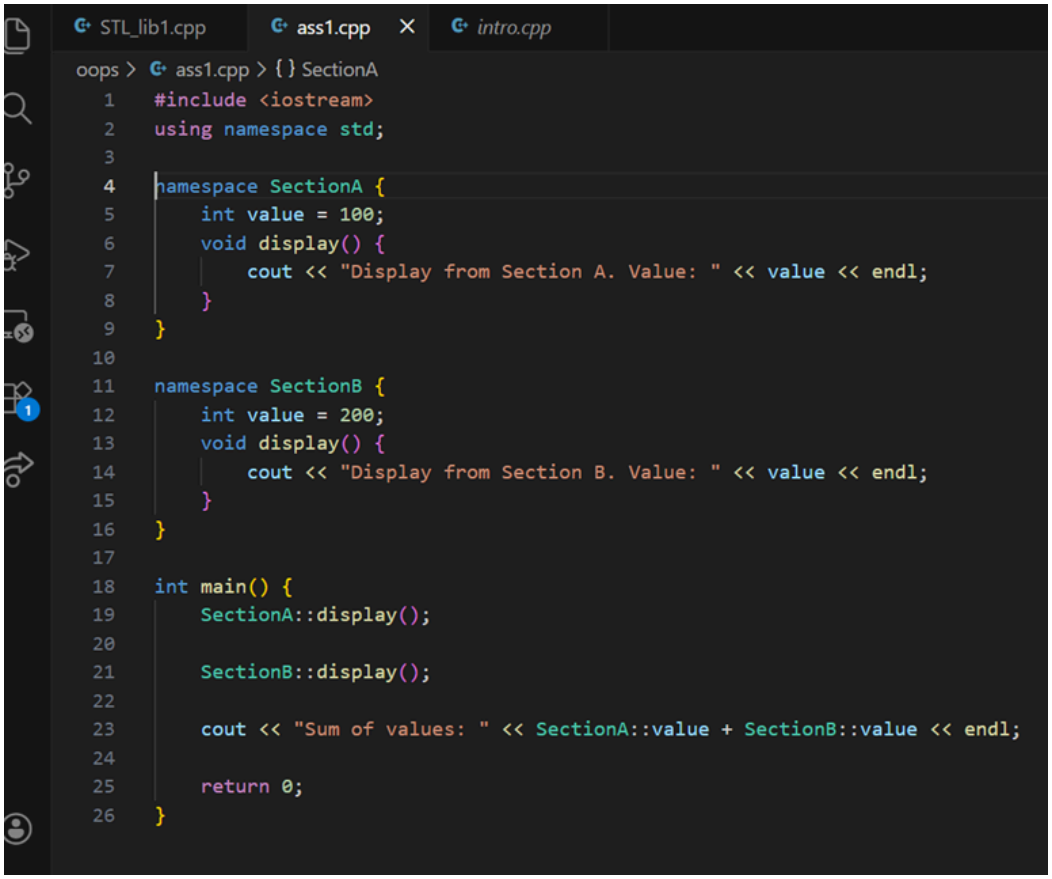
STL_lib1.cpp | ass1.cpp | intro.cpp

oops > ass1.cpp > { } SectionA

```cpp
#include <iostream>
using namespace std;

namespace SectionA {
    int value = 100;
    void display() {
        cout << "Display from Section A. Value: " << value << endl;
    }
}

namespace SectionB {
    int value = 200;
    void display() {
        cout << "Display from Section B. Value: " << value << endl;
    }
}

int main() {
    SectionA::display();

    SectionB::display();

    cout << "Sum of values: " << SectionA::value + SectionB::value << endl;

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\oops> cd "c:\
ass1 }
Display from Section A. Value: 100
Display from Section B. Value: 200
Sum of values: 300
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\oops>
```



6. Write a program to define a structure *Student* that contains 3 variables and 2 functions:
   - *Variables:*
     - *(a) name*
     - *(b) roll*
     - *(c) marks*
   - *Functions:*
     - *(1) setStudentData()*
     - *(2) getStudentData()*

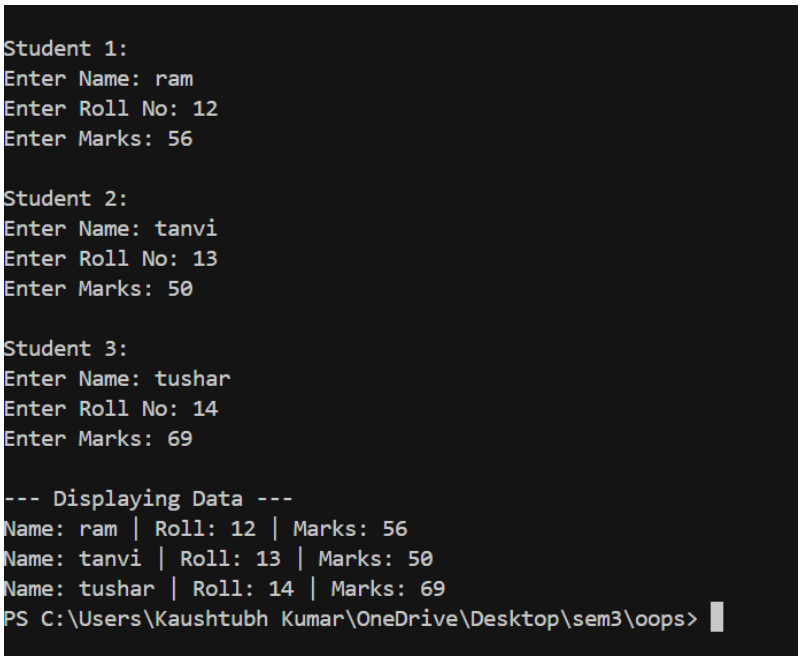
Use loops to input and output data for 3 student object entries.

STL_lib1.cpp | ass1.cpp | intro.cpp

oops > ass1.cpp > Student

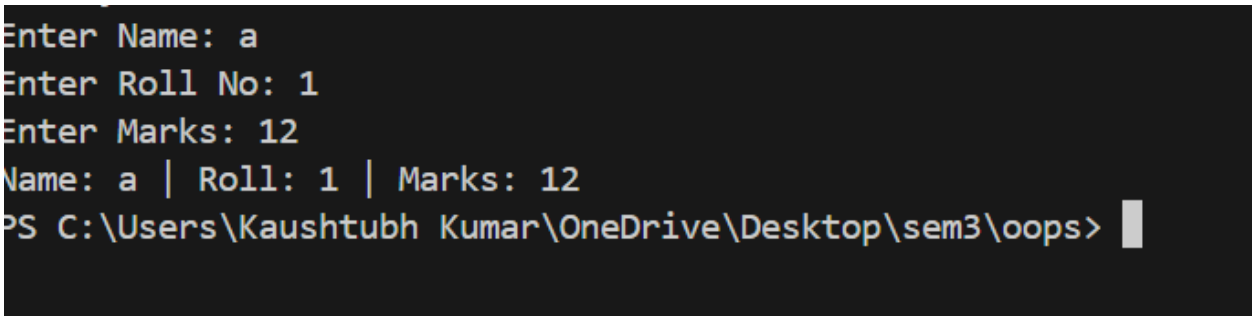
```cpp
#include <iostream>
using namespace std;

struct Student {
    string name;
    int roll;
    float marks;

    void setStudentData() {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Roll No: ";
        cin >> roll;
        cout << "Enter Marks: ";
        cin >> marks;
    }

    void getStudentData() {
        cout << "Name: " << name << " | Roll: " << roll << " | Marks: " << marks << endl;
    }
};

int main() {
    Student students[3];

    cout << "--- Enter Data for 3 Students ---" << endl;
    for(int i = 0; i < 3; i++) {
        cout << "\nStudent " << i+1 << ":" << endl;
        students[i].setStudentData();
    }

    cout << "\n--- Displaying Data ---" << endl;
    for(int i = 0; i < 3; i++) {
        students[i].getStudentData();
    }

    return 0;
}
```

OUTPUT:



7. In the previous program make the data private, and functions public.

OUTPUT:



8. Convert program #4 from struct to class.

KAUSHTUBH KUMAR 1024150191 2O23 OOPS_LAB

STL_lib1.cpp | ass1.cpp | intro.cpp

oops > ass1.cpp > main()

```cpp
#include <iostream>
using namespace std;

class Student {
string name;
int roll;
float marks;

    public:
     void setStudentData() {
        cout << "Enter Name: ";
            cin >> name;
            cout << "Enter Roll No: ";
            cin >> roll;
            cout << "Enter Marks: ";
            cin >> marks;
        }

        void getStudentData() {
            cout << "Name: " << name << " | Roll: " << roll << " | Marks: " << marks << endl;
        }
};

int main() {
    Student students[3];
    cout << "Enter Data for 3 Students (Using Class)" << endl;
    for(int i=0;i<3;i++) {
        cout <<"\nStudent "<< i+1 << ":"<< endl;
        students[i].setStudentData();
    }

    cout << "\n Displaying Data" << endl;
    for(int i = 0; i < 3; i++) {
        students[i].getStudentData();
    }
    return 0;
}
```

OUTPUT:

```
Student 1:
Enter Name: aashu
Enter Roll No: 1
Enter Marks: 0

Student 2:
Enter Name: rashu
Enter Roll No: 2
Enter Marks: 12

Student 3:
Enter Name: rishu
Enter Roll No: 3
Enter Marks: 34

 Displaying Data
Name: aashu | Roll: 1 | Marks: 0
Name: rashu | Roll: 2 | Marks: 12
Name: rishu | Roll: 3 | Marks: 34
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\oops>
```

10. WAP to read an integer value from the keyboard and display on screen “WELL DONE” that many times.

STL_lib1.cpp | ass1.cpp | intro.cpp

oops > ass1.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main() {
int n;
cin>>n;

    cout << "\n Displaying Data" << endl;
    for(int i=0;i<n; i++) {
        cout<<"WELL DONE \n";
    }
    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\oops> cd "
ass1 }
6

 Displaying Data
WELL DONE
WELL DONE
WELL DONE
WELL DONE
WELL DONE
WELL DONE
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\oops>
```

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

LAB ASSIGNMENT-2

Submitted by:

Kaushtubh Kumar

1024150191

2O23

1.Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members (properties):

- **Name**
- **Roll No**
- **Degree**
- **Hostel**
- **Current CGPA**

Member Functions (behavior):

1. **addDetails()**
2. **updateDetails()**
3. **updateCGPA()**
4. **updateHostel()**
5. **displayDetails()**

CODE

```
#include <iostream>
using namespace std;

struct Students
```

```
{
    string Name;
    int rollno;
    string degree;
    string hostel;
    float currentCGPA;

public:
    void add_details(){
        cout<<"enter name :";
        cin>>Name;

        cout<<"enter rollno :";
        cin>>rollno;

        cout<<"enter degree :";
        cin>>degree;

        cout<<"enter hostel :";
        cin>>hostel;

        cout<<"enter currentCGPA :";
        cin>>currentCGPA;
    }

    void display_details(){
        cout<<"entered name\t"<<Name<<endl;
        cout<<"entered rollno\t"<<rollno<<endl;
        cout<<"entered degree\t"<<degree<<endl;
        cout<<"entered hostel\t"<<hostel<<endl;
        cout<<"entered cgpa\t"<<currentCGPA<<endl;
    }

    void update_details(){
        cout<<"ENTER NEW NAME :";
        cin>>Name;

        cout<<"ENTER NEW ROLLNO :";
        cin>>rollno;

        cout<<"ENTER NEW DEGREE :";
        cin>>degree;

        cout<<"ENTER NEW HOSTEL :";
        cin>>hostel;

        cout<<"ENTER NEW CGPA :";
        cin>>currentCGPA;
    }

    void update_cgpa(){
        cout<<"UPDATE CGPA :" ;
        cin>>currentCGPA;
```

```
    }

    void update_hostel(){
        cout<<"UPDATE HOSTEL :";
        cin>>hostel;
    }

};

int main(){
    int n;
    cout<<"ENTER NO OF STUDENTS FOR ENTRY: ";
    cin>>n;
         Students s[n];

     for(int i = 0; i < n; i++){
        cout << "\n--- ENTER DETAILS OF STUDENTS " << i+1 << " ---\n";
        s[i].add_details();
    }

    for(int i = 0; i < n; i++){
        cout << "\n DETAILS OF STUDENTS " << i+1 << " ---\n";
        s[i].display_details();
    }

         Students x;
         x.add_details();
         x.update_details();
         x.update_cgpa();
         x.update_hostel();
         x.display_details();

    return 0;
}
```

OUTPUT

```
 oops.cpp -o oops } ; if ($?) { .\oops }
ENTER NO OF STUDENTS FOR ENTRY: 2

--- ENTER DETAILS OF STUDENTS 1 ---
enter name :scot
enter rollno :1
enter degree :CS
enter hostel :A
enter currentCGPA :5.5

--- ENTER DETAILS OF STUDENTS 2 ---
enter name :James
enter rollno :2
enter degree :AI
enter hostel :B
enter currentCGPA :8.3

 DETAILS OF STUDENTS 1 ---
entered name    scot
entered rollno  1
entered degree  CS
entered hostel  A
entered cgpa    5.5

 DETAILS OF STUDENTS 2 ---
entered name    James
entered rollno  2
entered degree  AI
entered hostel  B
entered cgpa    8.3
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

```
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques> cd "
 oops.cpp -o oops } ; if ($?) { .\oops }
enter name :rashu
enter rollno :123
enter degree :enc
enter hostel :K
enter currentCGPA :6.9
ENTER NEW NAME :costubh
ENTER NEW ROLLNO :123
ENTER NEW DEGREE :coe
ENTER NEW HOSTEL :M
ENTER NEW CGPA :7.9
UPDATE CGPA :8.1
UPDATE HOSTEL :X
entered name    costubh
entered rollno  123
entered degree  coe
entered hostel  X
entered cgpa    8.1
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

Q2.Differentiate between private and public access / scope. Perform Question No. 1 using class instead of structure, having the data members private and some member functions in private scope and some in public scope

Public: Members are accessible from outside the class (e.g., in main).

Private: Members are only accessible within the class itself. They are hidden from the outside world to ensure data security (Encapsulation).

CODE

```
#include <iostream>
#include <string>
using namespace std;

class StudentClass {
private:
    string Name;
    int RollNo;
    string Degree;
    string Hostel;
    float CurrentCGPA;

    void access_pvt() {
        cout<<endl;
        cout << "Accessed a private function\n";
    }

public:
    void addDetails() {
        cout << "Enter Name: "; cin >> Name;
        cout << "Enter Roll No: "; cin >> RollNo;
        cout << "Enter Degree: "; cin >> Degree;
        cout << "Enter Hostel: "; cin >> Hostel;
        cout << "Enter CGPA: "; cin >> CurrentCGPA;

        access_pvt();
    }

      void update_details(){
        cout<<"ENTER NEW NAME :";
        cin>>Name;

        cout<<"ENTER NEW ROLLNO :";
        cin>>RollNo;
```

```
        cout<<"ENTER NEW DEGREE :";
        cin>>Degree;

        cout<<"ENTER NEW HOSTEL :";
        cin>>Hostel;

        cout<<"ENTER NEW CGPA :";
        cin>>CurrentCGPA;
    }


    void updateCGPA() {
        cout << "Enter Updated CGPA: "; cin >> CurrentCGPA;
    }

    void updateHostel() {
        cout << "Enter Updated Hostel: "; cin >> Hostel;
    }

    void displaydetails() {
        cout << "\nStudent Details (Class)\n";
        cout << "Name: " << Name << "\nRoll No: " << RollNo
             << "\nDegree: " << Degree << "\nHostel: " << Hostel
             << "\nCGPA: " << CurrentCGPA << endl;
    }
};

int main() {
    StudentClass s1;
    s1.addDetails();
    s1.displaydetails();
    return 0;
}
```

OUTPUT

```
 oops.cpp -o oops } ; if ($?) { .\oops }
Enter Name: A
Enter Roll No: 1
Enter Degree: EEE
Enter Hostel: S
Enter CGPA: 8.6

Accessed a private function

Student Details (Class)
Name: A
Roll No: 1
Degree: EEE
Hostel: S
CGPA: 8.6
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

Q3.Create a code snippet that illustrates the following:
Calling of private member functions inside public member function.

CODE

```
#include <iostream>
using namespace std;

class Alpha {
private:
    void call_privateFunction() {
        cout << "Inside Private function" << endl;
    }

public:
    void publicFunction() {
        cout << "Inside Public function" << endl;
        call_privateFunction();
    }
};

int main() {
    Alpha x;
    x.publicFunction();
    return 0;
```

```
}
```

OUTPUT

```
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
 oops.cpp -o oops } ; if ($?) { .\oops }
Inside Public function
Inside Pruvate function
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

Q4.Define a class Rectangle with variables width and height of integer type along with the following:

(a) void getdata() to initialize object values

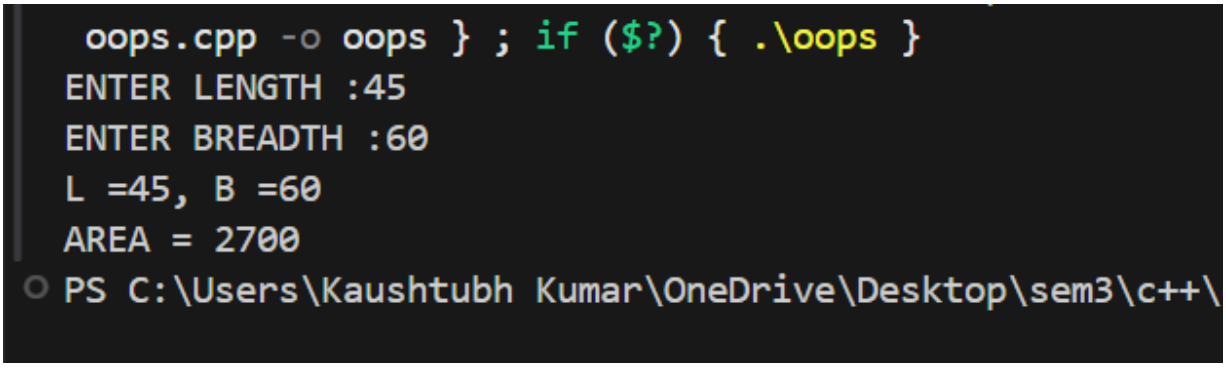
(b) void calculateArea() to calculate and display the area

CODE

```
#include <iostream>
using namespace std;
#include <vector>

class rectangle{
    private:
    float length;
    float breadth;

public:
void get_data(){
    cout<<"ENTER LENGTH :";
    cin>>length;
    cout<<"ENTER BREADTH :";
    cin>>breadth;
    cout<<"L ="<<length<<", B ="<<breadth<<endl;
}
```

```
float calculate_area(){

    return length*breadth;
}
};

int main(){
rectangle r;
r.get_data();
cout<<"AREA = "<<r.calculate_area()<<endl;
}
```

OUTPUT



Q5.Define a class Complex with variables real and imaginary along with the following:

(a) void setComplex(float, float) to initialize object values

(b) void displayComplex() to show the complex number

(c) Pass andS and return objects to calculate sum of two complex numbers. Display the sum.

CODE

```
#include <iostream>
using namespace std;

class Complex {
    float real;
    float imaginary;

public:
    void setComplex(float r, float i) {
        real = r;
        imaginary = i;
    }

    void displayComplex() {
        cout << real << " + " << imaginary << "i" << endl;
    }

    Complex sum(Complex c2) {
        Complex temp;
        temp.real = real + c2.real;
        temp.imaginary = imaginary + c2.imaginary;
        return temp;
    }
};

int main() {
    Complex c1, c2, c3;

    c1.setComplex(3.5, 2.5);
    c2.setComplex(1.5, 4.5);

    cout << "C1: "; c1.displayComplex();
    cout << "C2: "; c2.displayComplex();

    c3 = c1.sum(c2);

    cout << "Sum: "; c3.displayComplex();
    return 0;
}
```

OUTPUT

```
 tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) {
C1: 3.5 + 2.5i
C2: 1.5 + 4.5i
Sum: 5 + 7i
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

Q6.Implement scope resolution operator :: for the following uses:

(a) Class functions defined outside the class

(b) Access a global variable with the same name as a local variable

(c) Access static variables

(d) Use inbuilt libraries (cin, cout) with scope resolution operator

CODE

```
#include <iostream>

int num = 100;

class Scopes {
    static int staticVar;
public:
    void show();
    static void showStatic() {
        std::cout << "Static Variable: " << staticVar << std::endl;
    }
};

int Scopes::staticVar = 50;

void Scopes::show() {
    std::cout << "Function defined outside class." << std::endl;
}

int main() {
    int num = 20;

    std::cout << "Local num: " << num << std::endl;

    std::cout << "Global num: " << ::num << std::endl;
```

```
    Scopes inst;
    inst.show();

    Scopes::showStatic();

    return 0;
}
```

OUTPUT

```
  oops.cpp -o oops } ; if ($?) { .\oops }
Local num: 20
Global num: 100
Function defined outside class.
Static Variable: 50
PS C:\Users\Kaushtubh Kumar\OneDrive\Desktop\sem3\c++\ques>
```

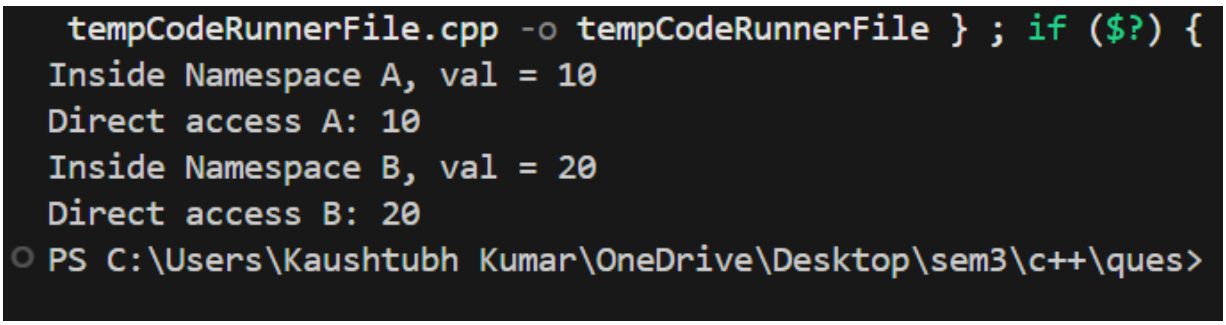
Q7.Implement *namespace* in a program to illustrate the use of same name variables and functions in different sections/libraries of the code.

CODE

```
#include <iostream>
using namespace std;

namespace NamespaceA {
    int val = 10;
    void show() {
        cout << "Inside Namespace A, val = " << val << endl;
    }
}
namespace NamespaceB {
    int val = 20;
    void show() {
        cout << "Inside Namespace B, val = " << val << endl;
```

```
    }
}

int main() {
    NamespaceA::show();
    cout << "Direct access A: " << NamespaceA::val << endl;

    NamespaceB::show();
    cout << "Direct access B: " << NamespaceB::val << endl;

    return 0;
}
```

OUTPUT



Lab Assignment 3

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Kaushtubh Kumar

Roll No. 1024150191

Batch: 2O23

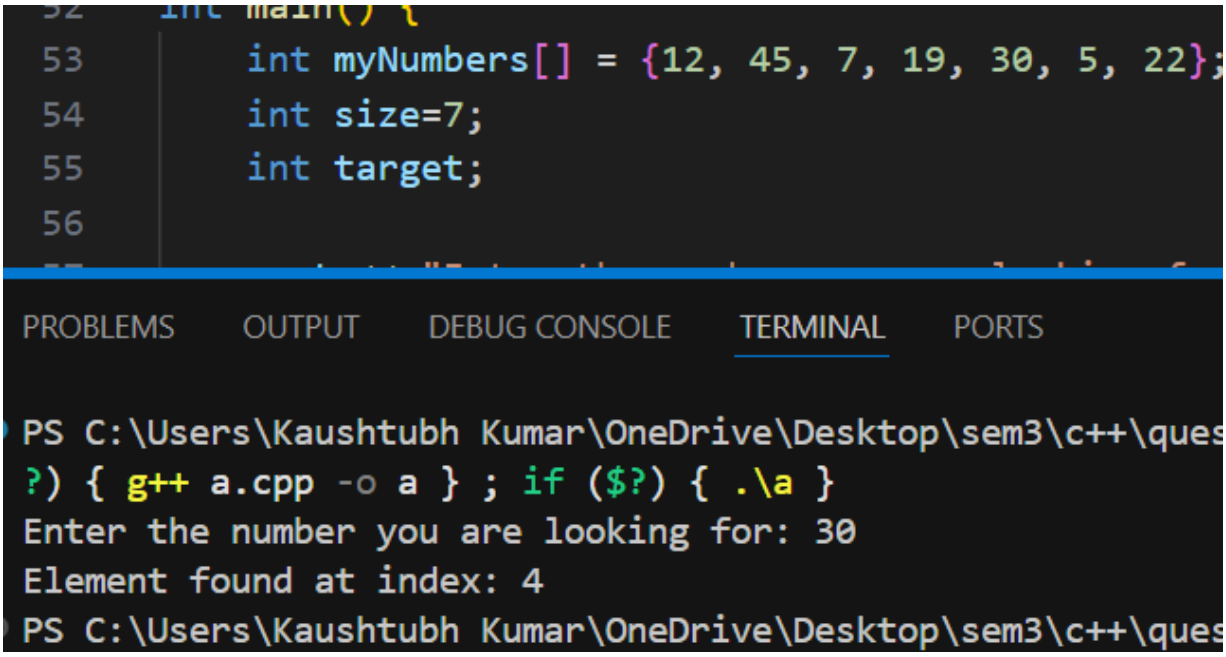
Q1 Write a program to implement (a) pointer to an object (b) this pointer. Practice both " (dot operator) and '->' (arrow operator).

CODE

```
#include <iostream>
using namespace std;

class ClassB;

class ClassA {
    int valA;
public:
    ClassA(int v) {
     valA = v;
    }
    friend void swapValues(ClassA&,ClassB&);
    void show() {
        cout << "ClassA: " << valA << endl;
    }
};

class ClassB {
    int valB;
public:
    ClassB(int v) : valB(v) {}
    friend void swapValues(ClassA &, ClassB &);
    void show() { cout << "ClassB: " << valB << endl; }
};

void swapValues(ClassA &a, ClassB &b) {
    int temp = a.valA;
    a.valA = b.valB;
    b.valB = temp;
}

int main() {
    ClassA obj1(10);
    ClassB obj2(20);

    cout<<"Before swap:"<<endl;
    obj1.show(); obj2.show();
```

```
    swapValues(obj1, obj2);

    cout<<"After swap:"<<endl;
    obj1.show();
    obj2.show();
    return 0;
}
```

TESTCASE



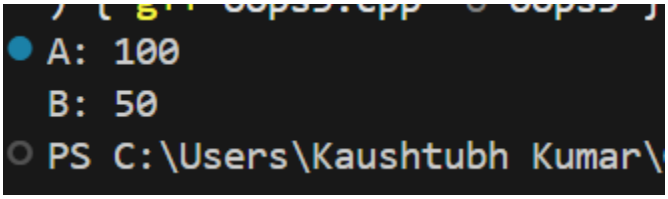
Q2. Write a program to swap private values of two classes using a friend function.

CODE

```
#include <iostream>
using namespace std;

class ClassB; // Forward declaration

class ClassA {
    int a;
public:
```

```
    ClassA(int v) : a(v) {}
    void show() { cout << "A: " << a << endl; }
    friend void swap(ClassA &objA, ClassB &objB);
};

class ClassB {
    int b;
public:
    ClassB(int v) : b(v) {}
    void show() { cout << "B: " << b << endl; }
    friend void swap(ClassA &objA, ClassB &objB);
};

void swap(ClassA &objA, ClassB &objB) {
    int temp = objA.a;
    objA.a = objB.b;
    objB.b = temp;
}

int main() {
    ClassA x(50);
    ClassB y(100);
    swap(x, y);
    x.show(); y.show();
    return 0;
}
```

TESTCASE



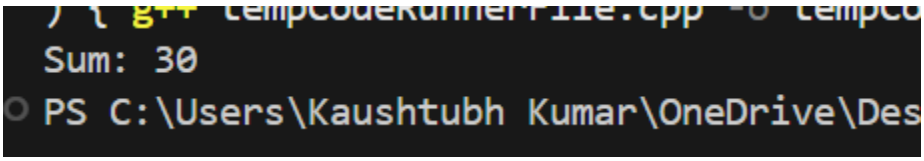
Q3. Write a program to add data objects of two different classes using friend functions.

CODE

```
#include <iostream>
```

```
using namespace std;

class ClassY;

class ClassX {
    int data;
public:
    ClassX(int d) : data(d) {}
    friend void sum(ClassX, ClassY);
};

class ClassY {
    int data;
public:
    ClassY(int d) : data(d) {}
    friend void sum(ClassX, ClassY);
};

void sum(ClassX obj1, ClassY obj2) {
    cout << "Sum: " << obj1.data + obj2.data << endl;
}

int main() {
    ClassX a(10);
    ClassY b(20);
    sum(a, b);
    return 0;
}
```

TESTCASE



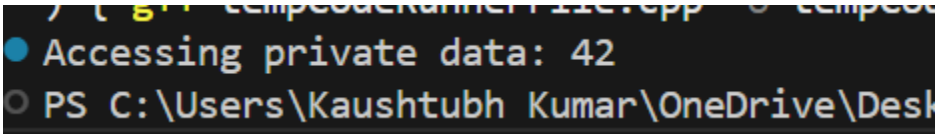
Q4. Write a program to demonstrate the working of friend class.

CODE

```
#include <iostream>
using namespace std;
```

```
class Base {
    int secret = 42;
    friend class Auditor;
};

class Auditor {
public:
    void reveal(Base &b) {
        cout << "Accessing private data: " << b.secret <<
endl;
    }
};

int main() {
    Base b;
    Auditor a;
    a.reveal(b);
    return 0;
}
```

TESTCASE



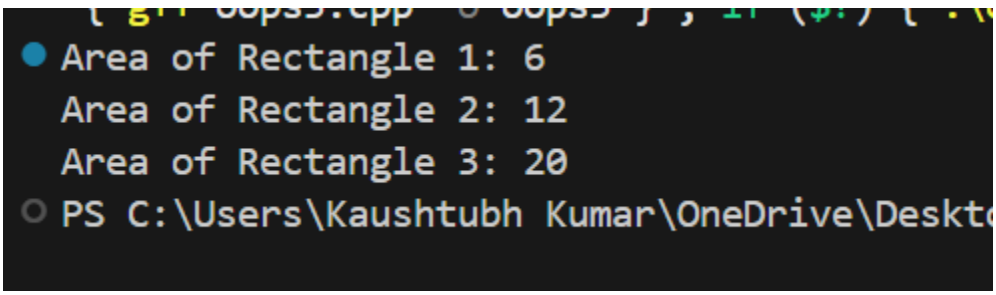
Q5 Write a program using Array of Objects to display area of multiple rectangles.

CODE

```
#include <iostream>
using namespace std;

class Rectangle {
    float l, b;
public:
    void setData(float length, float breadth) {
        l = length; b = breadth;
    }
```

```
    float getArea() { return l * b; }
};

int main() {
    Rectangle rects[3];
    for(int i = 0; i < 3; i++) {
        rects[i].setData(i + 2, i + 3);
        cout << "Area of Rectangle " << i+1 << ": " <<
rects[i].getArea() << endl;
    }
    return 0;
}
```

TESTCASE



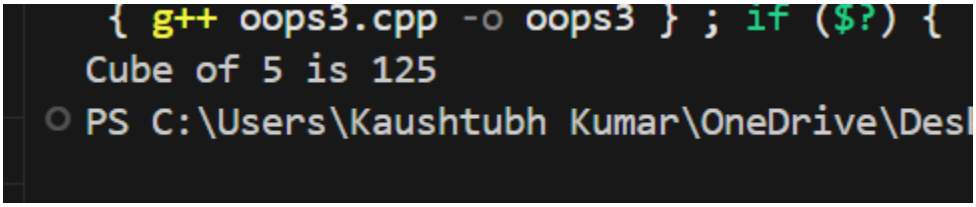
Q6. Write a program to define function cube() as inline for calculating cube of a number.

CODE

```
#include <iostream>
using namespace std;

inline int cube(int n) {
    return n * n * n;
}

int main() {
    int num = 5;
    cout << "Cube of " << num << " is " << cube(num) <<
endl;
    return 0;
```

```
}
```

TESTCASE



Q7 Write a program to pass an object as an argument and return the object from a function.

(a) Use pass-by-value

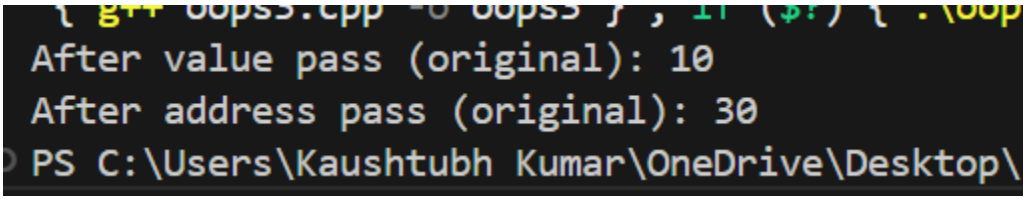
(b)Use pass-by-address

CODE

```
#include <iostream>
using namespace std;

class Box {
public:
    int value;
    Box(int v) : value(v) {}
};

Box processByValue(Box b) {
    b.value += 10;
    return b;
}

(Box* processByAddress(Box *b) {
    b->value += 20;
    return b;
}
```

```
int main() {
    Box myBox(10);


    Box result1 = processByValue(myBox);
    cout << "After value pass (original): " << myBox.value
<< endl; // Remains 10


    Box* result2 = processByAddress(&myBox);
    cout << "After address pass (original): " <<
myBox.value << endl; // Changes to 30

    return 0;
}
```

TESTCASE



Lab Assignment 4

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Kaushtubh Kumar

Roll No. 1024150191

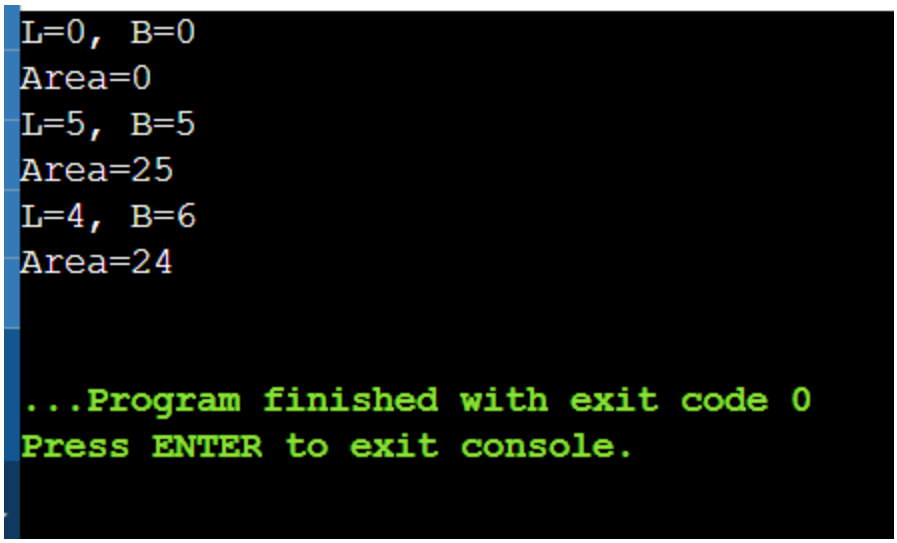
Batch: 2O23

Question 1

Create a class named 'Rectangle' with two data members- length and breadth and a function to calculate the area which is 'length*breadth'. The class has three constructors: (1) No parameter - assign zero, (2) Two parameters, (3) One parameter - both get same value. Create objects with none, one and two parameters and print their areas.

Code

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length;
    double breadth;
public:
    Rectangle() {
        length = 0;
        breadth = 0;
    }
    Rectangle(double l, double b) {
        length = l;
        breadth = b;
    }
    Rectangle(double side) {
        length = breadth = side;
    }
    double area() {
        return length * breadth;
    }
    void display() {
        cout << "L=" << length << ", B=" << breadth << endl;
    }
};

int main() {
    Rectangle r1;
    r1.display();
    cout << "Area=" << r1.area() << endl;

    Rectangle r2(5);
    r2.display();
    cout << "Area=" << r2.area() << endl;

    Rectangle r3(4, 6);
    r3.display();
    cout << "Area=" << r3.area() << endl;

    return 0;
}
```



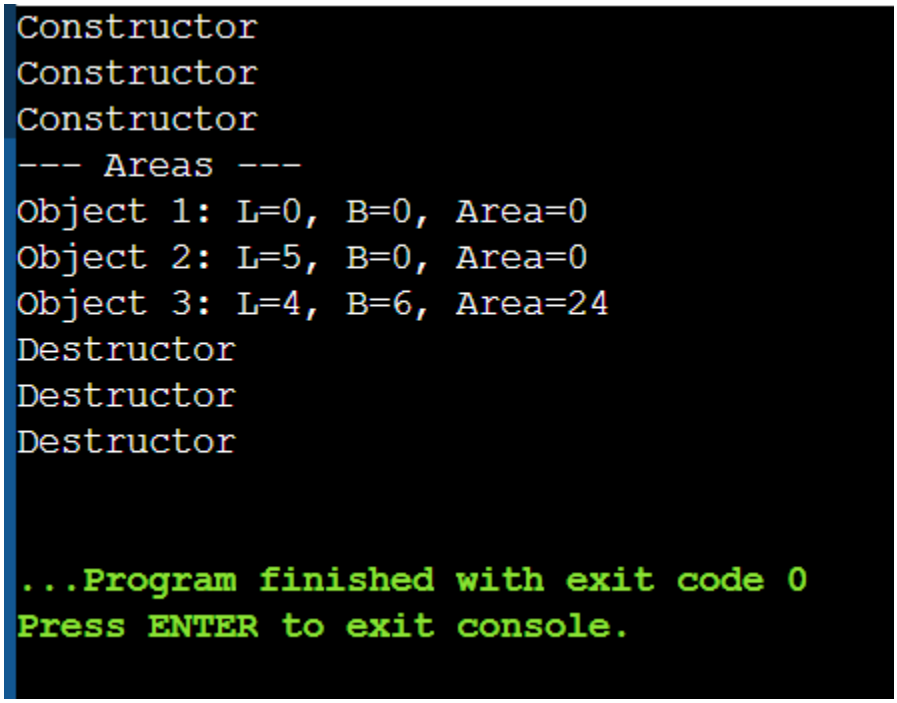
Question 2

Redefine the program with array of objects of Rectangle class, calculate area for each object. Implement constructors with default arguments and destructor.

Code

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length, breadth;
public:
    Rectangle(double l=0, double b=0) {
        length = l;
        breadth = b;
        cout << "Constructor" << endl;
    }
    ~Rectangle() {
        cout << "Destructor" << endl;
    }
    double area() {
        return length * breadth;
    }
    void display() {
        cout << "L=" << length
             << ", B=" << breadth
             << ", Area=" << area() << endl;
    }
};

int main() {
    Rectangle arr[3] = {
        Rectangle(),
        Rectangle(5),
```

```
        Rectangle(4, 6)
    };

    cout << "
--- Areas ---" << endl;
    for (int i = 0; i < 3; i++) {
        cout << "Object " << i+1 << ": ";
        arr[i].display();
    }
    return 0;
}
```



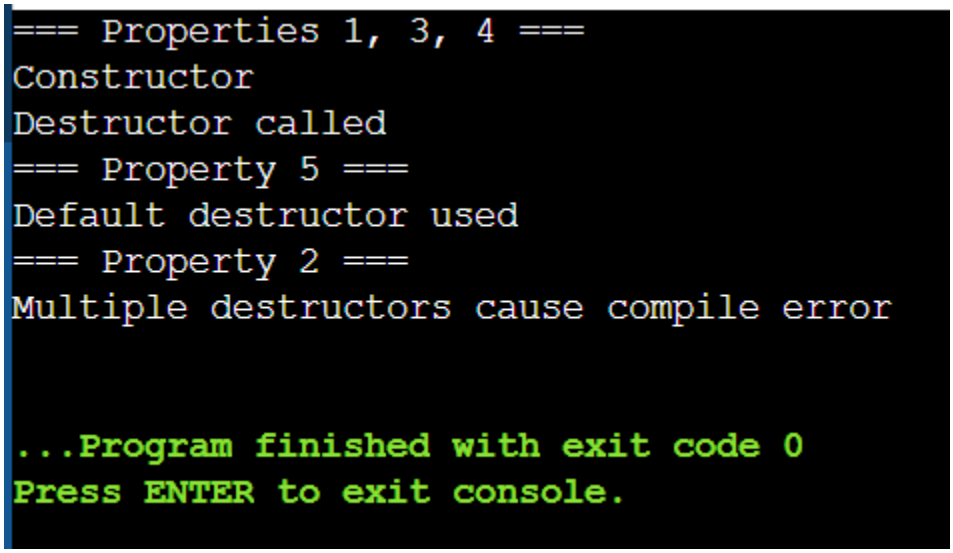
Question 3

Verify destructor properties: (1) Name begins with ~ and matches class name, (2) Only one destructor allowed, (3) No parameters, (4) No return type, (5) Compiler generates default when not specified.

Code

```
#include <iostream>
using namespace std;

class TestDestructor {
public:
    TestDestructor() {
        cout << "Constructor" << endl;
    }
    ~TestDestructor() {
        cout << "Destructor called" << endl;
    }
```

```
};

class NoDestructor {
    // No destructor defined
    // Compiler generates default
};

int main() {
    cout << "=== Properties 1, 3, 4 ===" << endl;
    {
        TestDestructor obj;
    }

    cout << "
=== Property 5 ===" << endl;
    {
        NoDestructor obj2;
        cout << "Default destructor used" << endl;
    }

    cout << "
=== Property 2 ===" << endl;
    cout << "Multiple destructors cause compile error" << endl;

    return 0;
}
```



Question 4

Implement dynamic memory allocation using new and delete for: integer variable, float variable, integer array, float array, class objects, array of objects.

Code

```
#include <iostream>
using namespace std;
```

```
class Demo {
public:
    int value;
    Demo(int v) {
        value = v;
        cout << "Created: " << value << endl;
    }
    ~Demo() {
        cout << "Destroyed: " << value << endl;
    }
};

int main() {
    // Integer variable
    int* ip = new int;
    *ip = 42;
    cout << "int: " << *ip << endl;
    delete ip;

    // Float variable
    float* fp = new float;
    *fp = 3.14f;
    cout << "float: " << *fp << endl;
    delete fp;

    // Integer array
    int* ia = new int[5]{10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        cout << ia[i] << " ";
    cout << endl;
    delete[] ia;

    // Float array
    float* fa = new float[4]{1.5f, 2.5f, 3.5f, 4.5f};
    for (int i = 0; i < 4; i++)
        cout << fa[i] << " ";
    cout << endl;
    delete[] fa;

    // Class object
    Demo* obj = new Demo(100);
    delete obj;

    // Array of objects
    Demo* arr = new Demo[3];
    arr[0] = Demo(1);
    arr[1] = Demo(2);
    arr[2] = Demo(3);
    delete[] arr;

    cout << "All memory freed!" << endl;
    return 0;
}
```

```
Destroyed: 100
Created default: 0
Created default: 0
Created default: 0
Created: 1
Destroyed: 1
Created: 2
Destroyed: 2
Created: 3
Destroyed: 3
Destroyed: 3
Destroyed: 2
Destroyed: 1
All memory freed!


...Program finished with exit code 0
Press ENTER to exit console.
```

Lab Assignment 5

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Kaushtubh Kumar

Roll No. 1024150191

Batch: 2O23

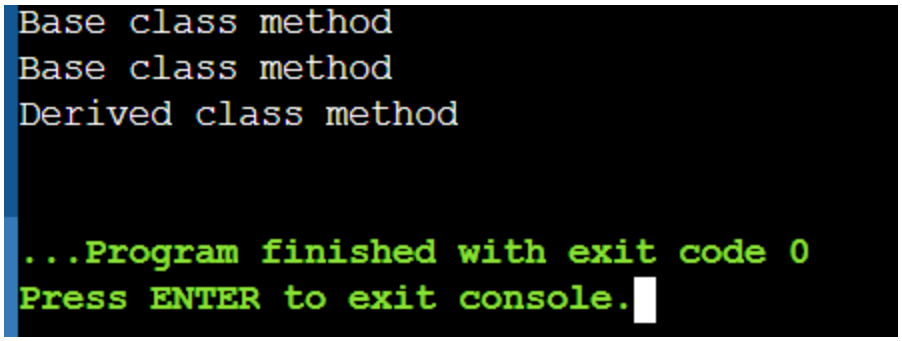
Question 1

Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

Code

```
#include <iostream>
using namespace std;

class Base {
public:
    void displayBase() {
        cout << "Base class method" << endl;
    }
};

class Derived : public Base {
public:
    void displayDerived() {
        cout << "Derived class method" << endl;
    }
};

int main() {
    Base b;
    b.displayBase();

    Derived d;
    d.displayBase();      // Inherited from Base
    d.displayDerived();   // Own method

    return 0;
}
```

OUTPUT



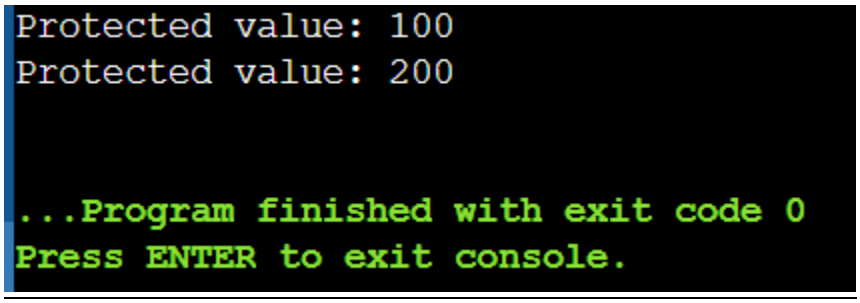
Question 2

Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

Code

```
#include <iostream>
using namespace std;

class Base {
protected:
    int protectedValue;
public:
    Base() : protectedValue(100) {}
    void setProtectedValue(int val) {
        protectedValue = val;
    }
};

class Derived : public Base {
public:
    void displayProtected() {
        cout << "Protected value: " << protectedValue << endl;
    }
    void modifyProtected(int val) {
        protectedValue = val;
    }
};

int main() {
    Derived d;
    d.displayProtected();
    d.modifyProtected(200);
    d.displayProtected();
    return 0;
}
```

OUTPUT



Question 3

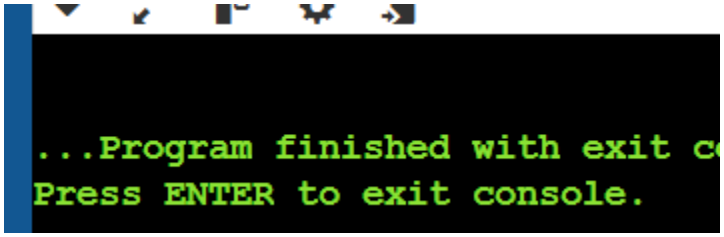
Try protected and private access modifiers to understand the difference of various modes of inheritance.

Code

```
#include <iostream>
using namespace std;

class Base {
public:
    int publicVar;
protected:
    int protectedVar;
private:
    int privateVar;
public:
    Base() : publicVar(1), protectedVar(2), privateVar(3) {}
};

class DerivedPublic : public Base {
    // public members stay public
    // protected members stay protected
    // private members not accessible
};

class DerivedProtected : protected Base {
    // public members become protected
    // protected members stay protected
};

class DerivedPrivate : private Base {
    // public members become private
    // protected members become private
};

int main() {
    DerivedPublic dp;
    dp.publicVar = 10;  // Accessible
```

```
    DerivedProtected dpr;
    // dpr.publicVar = 10;  // Error: now protected

    DerivedPrivate dpv;
    // dpv.publicVar = 10;  // Error: now private

    return 0;
}
```

OUTPUT



Question 4

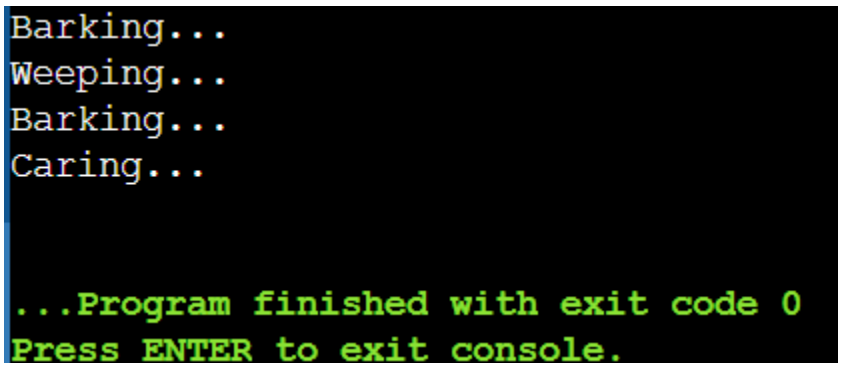
Write small C++ codes for each inheritance type (Single, Multiple, Hierarchical, Multilevel, Hybrid).

Code

```
#include <iostream>
using namespace std;

// Single Inheritance
class Animal {
public:
    void eat() { cout << "Eating..." << endl; }
};
class Dog : public Animal {
public:
    void bark() { cout << "Barking..." << endl; }
};

// Multiple Inheritance
class Walk {
public:
    void walk() { cout << "Walking..." << endl; }
};
class Swim {
public:
    void swim() { cout << "Swimming..." << endl; }
};
class Frog : public Walk, public Swim {};

// Hierarchical Inheritance
```

```
class Bird : public Animal {
public:
    void fly() { cout << "Flying..." << endl; }
};
class Fish : public Animal {
public:
    void swim() { cout << "Swimming..." << endl; }
};

// Multilevel Inheritance
class Puppy : public Dog {
public:
    void weep() { cout << "Weeping..." << endl; }
};

// Hybrid Inheritance
class Pet {
public:
    void care() { cout << "Caring..." << endl; }
};
class PetDog : public Dog, public Pet {};

int main() {
    Dog d; d.eat(); d.bark();
    Frog f; f.walk(); f.swim();
    Bird b; b.eat(); b.fly();
    Puppy p; p.eat(); p.bark(); p.weep();
    PetDog pd; pd.bark(); pd.care();
    return 0;
}
```

OUTPUT



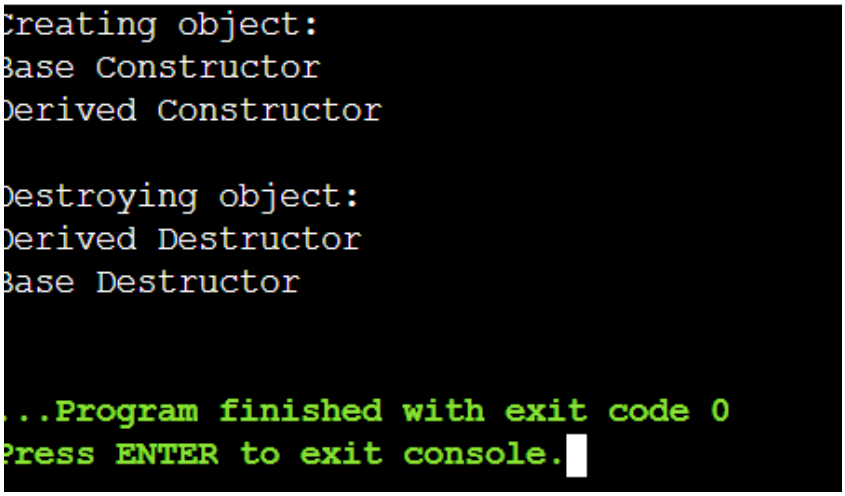
Question 5

How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

Code

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() { cout << "Base Constructor" << endl; }
    ~Base() { cout << "Base Destructor" << endl; }
};

class Derived : public Base {
public:
    Derived() { cout << "Derived Constructor" << endl; }
    ~Derived() { cout << "Derived Destructor" << endl; }
};

int main() {
    cout << "Creating object:" << endl;
    Derived d;
    cout << "\nDestroying object:" << endl;
    return 0;
}
```

OUTPUT



Question 6

Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Book {
protected:
    string title;
    string author;
    double price;
public:
    Book(string t, string a, double p)
        : title(t), author(a), price(p) {}
    void display() {
        cout << "Title: " << title
             << ", Author: " << author
             << ", Price: $" << price << endl;
    }
};

class Textbook : public Book {
private:
    string subject;
public:
    Textbook(string t, string a, double p, string s)
        : Book(t, a, p), subject(s) {}
    void displayTextbook() {
        display();
        cout << "Subject: " << subject << endl;
    }
};

int main() {
    Book b("1984", "George Orwell", 15.99);
    b.display();

    Textbook tb("Physics Vol 1", "H.C. Verma", 29.99, "Physics");
    tb.displayTextbook();

    return 0;
}
```

OUTPUT

```
Title: 1984, Author: George Orwell, Price: $15.99
Title: Physics Vol 1, Author: H.C. Verma, Price: $29.99
Subject: Physics


...Program finished with exit code 0
Press ENTER to exit console.
```

Question 7

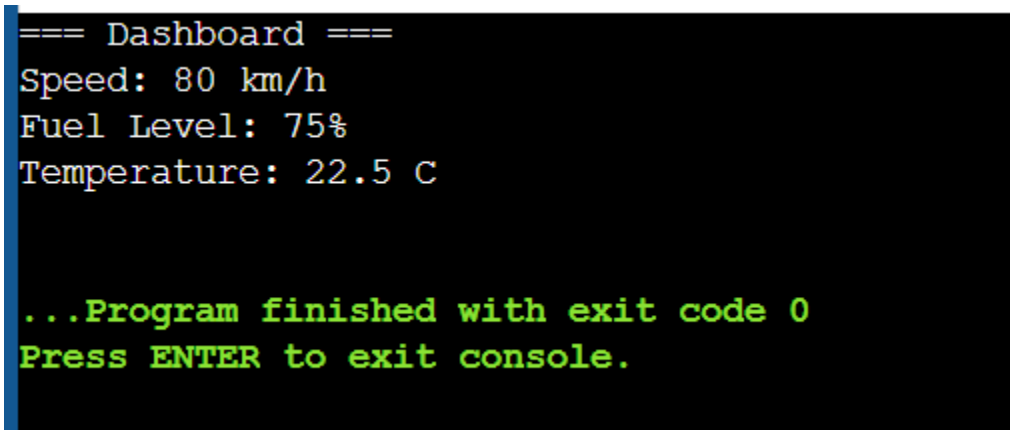
Implement CarDashboard class that inherits from Speedometer, FuelGauge, and Thermometer (Multiple Inheritance).

Code

```
#include <iostream>
#include <string>
using namespace std;

class Speedometer {
protected:
    int speed;
public:
    Speedometer(int s = 0) : speed(s) {}
    void setSpeed(int s) { speed = s; }
    int getSpeed() { return speed; }
    void displaySpeed() {
        cout << "Speed: " << speed << " km/h" << endl;
    }
};

class FuelGauge {
protected:
    int fuelLevel;
public:
    FuelGauge(int f = 100) : fuelLevel(f) {}
    void setFuelLevel(int f) { fuelLevel = f; }
    int getFuelLevel() { return fuelLevel; }
    void displayFuel() {
        cout << "Fuel Level: " << fuelLevel << "%" << endl;
    }
};

class Thermometer {
protected:
    double temperature;
public:
    Thermometer(double t = 25.0) : temperature(t) {}
    void setTemperature(double t) { temperature = t; }
    double getTemperature() { return temperature; }
    void displayTemp() {
```

```
        cout << "Temperature: " << temperature << " C" << endl;
    }
};

class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    CarDashboard(int s, int f, double t)
        : Speedometer(s), FuelGauge(f), Thermometer(t) {}
    void displayAll() {
        cout << "=== Dashboard ===" << endl;
        displaySpeed();
        displayFuel();
        displayTemp();
    }
};

int main() {
    CarDashboard cd(80, 75, 22.5);
    cd.displayAll();
    return 0;
}
```

OUTPUT



Question 8

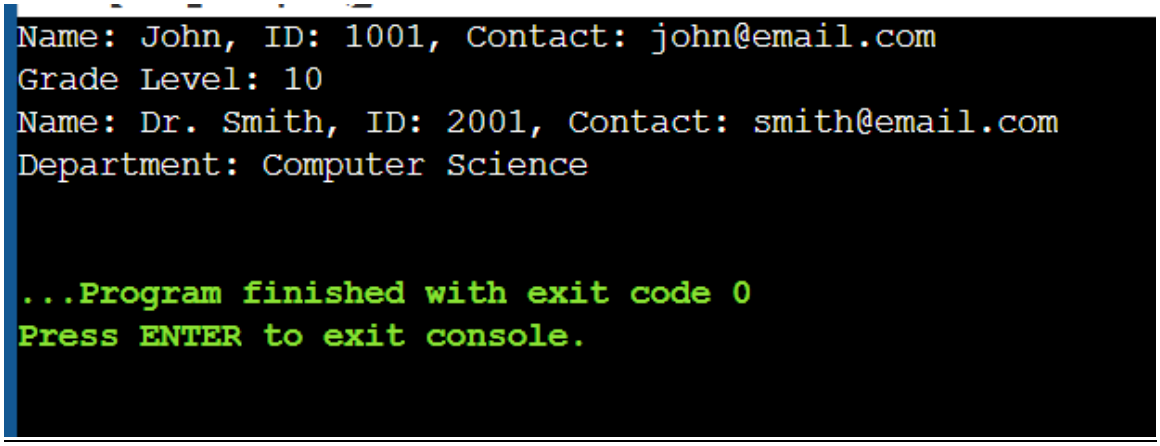
Implement hierarchical inheritance with LibraryUser base class and Student, Teacher derived classes.

Code

```
#include <iostream>
#include <string>
using namespace std;

class LibraryUser {
protected:
    string name;
    int id;
    string contact;
public:
    LibraryUser(string n, int i, string c)
        : name(n), id(i), contact(c) {}
    void displayUser() {
        cout << "Name: " << name
             << ", ID: " << id
             << ", Contact: " << contact << endl;
    }
};

class Student : public LibraryUser {
private:
    int gradeLevel;
public:
    Student(string n, int i, string c, int g)
        : LibraryUser(n, i, c), gradeLevel(g) {}
    void displayStudent() {
        displayUser();
        cout << "Grade Level: " << gradeLevel << endl;
    }
};

class Teacher : public LibraryUser {
private:
    string department;
public:
    Teacher(string n, int i, string c, string d)
        : LibraryUser(n, i, c), department(d) {}
    void displayTeacher() {
        displayUser();
        cout << "Department: " << department << endl;
    }
};

int main() {
    Student s("John", 1001, "john@email.com", 10);
    s.displayStudent();
```

```
    Teacher t("Dr. Smith", 2001, "smith@email.com", "Computer Science");
    t.displayTeacher();

    return 0;
}
```

OUTPUT



Question 9

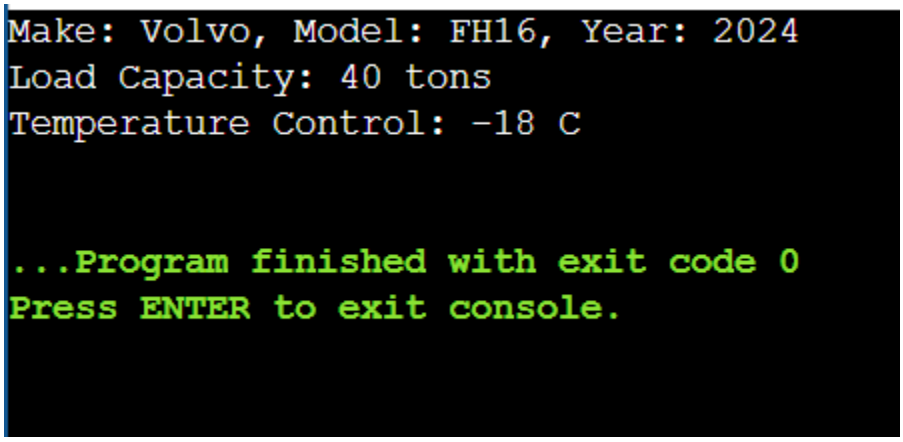
Implement multilevel inheritance with Vehicle -> Truck -> RefrigeratedTruck.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
protected:
    string make;
    string model;
    int year;
public:
    Vehicle(string mk, string md, int y)
        : make(mk), model(md), year(y) {}
    void displayVehicle() {
        cout << "Make: " << make
             << ", Model: " << model
             << ", Year: " << year << endl;
    }
};

class Truck : public Vehicle {
protected:
    double loadCapacity;
public:
```

```
    Truck(string mk, string md, int y, double lc)
        : Vehicle(mk, md, y), loadCapacity(lc) {}
    void displayTruck() {
        displayVehicle();
        cout << "Load Capacity: " << loadCapacity << " tons" << endl;
    }
};

class RefrigeratedTruck : public Truck {
private:
    double temperatureControl;
public:
    RefrigeratedTruck(string mk, string md, int y, double lc, double tc)
        : Truck(mk, md, y, lc), temperatureControl(tc) {}
    void displayRefrigeratedTruck() {
        displayTruck();
        cout << "Temperature Control: " << temperatureControl << " C" << endl;
    }
};

int main() {
    RefrigeratedTruck rt("Volvo", "FH16", 2024, 40.0, -18.0);
    rt.displayRefrigeratedTruck();
    return 0;
}
```

OUTPUT



Question 10

Implement hybrid inheritance with Person -> Staff, Student and TeachingAssistant inheriting from both Staff and Student.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    string address;
public:
    Person(string n, string a) : name(n), address(a) {}
    void displayPerson() {
        cout << "Name: " << name
             << ", Address: " << address << endl;
    }
};

class Staff : virtual public Person {
protected:
    int employeeId;
    string department;
public:
    Staff(string n, string a, int id, string d)
        : Person(n, a), employeeId(id), department(d) {}
    void displayStaff() {
        displayPerson();
        cout << "Employee ID: " << employeeId
             << ", Department: " << department << endl;
    }
};

class Student : virtual public Person {
protected:
    int studentId;
    string grade;
public:
    Student(string n, string a, int id, string g)
        : Person(n, a), studentId(id), grade(g) {}
    void displayStudent() {
        displayPerson();
        cout << "Student ID: " << studentId
             << ", Grade: " << grade << endl;
    }
};

class TeachingAssistant : public Staff, public Student {
public:
    TeachingAssistant(string n, string a, int eid, string d, int sid, string g)
```

```
        : Person(n, a), Staff(n, a, eid, d), Student(n, a, sid, g) {}
    void displayTA() {
        displayPerson();
        cout << "Employee ID: " << employeeId
             << ", Department: " << department << endl;
        cout << "Student ID: " << studentId
             << ", Grade: " << grade << endl;
    }
};

int main() {
    TeachingAssistant ta("Alice", "123 Main St", 5001, "CS", 6001, "A");
    ta.displayTA();
    return 0;
}
```

OUTPUT

```
Name: Alice, Address: 123 Main St
Employee ID: 5001, Department: CS
Student ID: 6001, Grade: A


...Program finished with exit code 0
Press ENTER to exit console.
```

Lab Assignment 6

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Kaushtubh Kumar

Roll No. 1024150191

Batch: 2O23

Question 1

Implement a class Library that contains an array of Book objects. The Book class should have attributes like title, author, and ISBN. The Library class should provide methods: addNewBook, removeBooks, displayDetails.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Book {
public:
    string title;
    string author;
    string ISBN;
};

class Library {
private:
    Book books[100];
    int bookCount;
public:
    Library() : bookCount(0) {}

    bool addNewBook(string &title, string &author, string &ISBN) {
        if (bookCount >= 100) return false;
        books[bookCount].title = title;
        books[bookCount].author = author;
        books[bookCount].ISBN = ISBN;
        bookCount++;
        return true;
    }

    bool removeBooks(string &ISBN) {
        for (int i = 0; i < bookCount; i++) {
            if (books[i].ISBN == ISBN) {
                for (int j = i; j < bookCount - 1; j++) {
                    books[j] = books[j + 1];
                }
                bookCount--;
                return true;
            }
        }
        return false;
    }

    void displayDetails() {
        cout << "=== Library Books ===" << endl;
        for (int i = 0; i < bookCount; i++) {
            cout << "Book " << i + 1 << ":" << endl;
            cout << "  Title: " << books[i].title << endl;
            cout << "  Author: " << books[i].author << endl;
```

```
            cout << "  ISBN: " << books[i].ISBN << endl;
        }
    }
};

int main() {
    Library lib;

    string t1 = "The Great Gatsby", a1 = "F. Scott Fitzgerald", i1 = "978-
0743273565";
    string t2 = "1984", a2 = "George Orwell", i2 = "978-0451524935";
    string t3 = "To Kill a Mockingbird", a3 = "Harper Lee", i3 = "978-
0061120084";
    string t4 = "Pride and Prejudice", a4 = "Jane Austen", i4 = "978-
0141439518";
    string t5 = "The Catcher in the Rye", a5 = "J.D. Salinger", i5 = "978-
0316769488";

    lib.addNewBook(t1, a1, i1);
    lib.addNewBook(t2, a2, i2);
    lib.addNewBook(t3, a3, i3);
    lib.addNewBook(t4, a4, i4);
    lib.addNewBook(t5, a5, i5);

    lib.displayDetails();

    cout << "\n--- Removing Book ---" << endl;
    lib.removeBooks(i2);
    lib.displayDetails();

    return 0;
}
```

```
=== Library Books ===
Book 1:
  Title: The Great Gatsby
  Author: F. Scott Fitzgerald
  ISBN: 978-0743273565
Book 2:
  Title: 1984
  Author: George Orwell
  ISBN: 978-0451524935
Book 3:
  Title: To Kill a Mockingbird
  Author: Harper Lee
  ISBN: 978-0061120084
Book 4:
  Title: Pride and Prejudice
  Author: Jane Austen
  ISBN: 978-0141439518
Book 5:
  Title: The Catcher in the Rye
  Author: J.D. Salinger
  ISBN: 978-0316769488

--- Removing Book ---
=== Library Books ===
Book 1:
  Title: The Great Gatsby
  Author: F. Scott Fitzgerald
  ISBN: 978-0743273565
Book 2:
  Title: To Kill a Mockingbird
  Author: Harper Lee
  ISBN: 978-0061120084
Book 3:
  Title: Pride and Prejudice
  Author: Jane Austen
  ISBN: 978-0141439518
Book 4:
```

Question 2

Implement Question 1 using constructors. Add default constructor, parameterized constructor, and copy constructor. Use the this pointer instead of scope resolution operator.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Book {
public:
    string title;
    string author;
    string ISBN;

    Book() {
        this->title = "";
        this->author = "";
        this->ISBN = "";
    }

    Book(string title, string author, string ISBN) {
        this->title = title;
        this->author = author;
        this->ISBN = ISBN;
    }

    Book(const Book &other) {
        this->title = other.title;
        this->author = other.author;
        this->ISBN = other.ISBN;
    }
};

class Library {
private:
    Book books[100];
    int bookCount;
public:
    Library() : bookCount(0) {}

    bool addNewBook(string &title, string &author, string &ISBN) {
        if (bookCount >= 100) return false;
        books[bookCount] = Book(title, author, ISBN);
        bookCount++;
        return true;
    }

    bool removeBooks(string &ISBN) {
        for (int i = 0; i < bookCount; i++) {
            if (books[i].ISBN == ISBN) {
```

```
                for (int j = i; j < bookCount - 1; j++) {
                    books[j] = books[j + 1];
                }
                bookCount--;
                return true;
            }
        }
        return false;
    }

    void displayDetails() {
        cout << "=== Library Books ===" << endl;
        for (int i = 0; i < bookCount; i++) {
            cout << "Book " << i + 1 << ":" << endl;
            cout << "  Title: " << books[i].title << endl;
            cout << "  Author: " << books[i].author << endl;
            cout << "  ISBN: " << books[i].ISBN << endl;
        }
    }
};

int main() {
    // Array of objects using Initializer list
    Book bookList[] = {
        Book("The Great Gatsby", "F. Scott Fitzgerald", "978-0743273565"),
        Book("1984", "George Orwell", "978-0451524935"),
        Book("To Kill a Mockingbird", "Harper Lee", "978-0061120084")
    };

    // Dynamic initialization
    Book* dynamicBooks = new Book[2];
    dynamicBooks[0] = Book("Pride and Prejudice", "Jane Austen", "978-
0141439518");
    dynamicBooks[1] = Book("The Catcher in the Rye", "J.D. Salinger", "978-
0316769488");

    Library lib;
    string t1 = "The Great Gatsby", a1 = "F. Scott Fitzgerald", i1 = "978-
0743273565";
    string t2 = "1984", a2 = "George Orwell", i2 = "978-0451524935";
    string t3 = "To Kill a Mockingbird", a3 = "Harper Lee", i3 = "978-
0061120084";
    string t4 = "Pride and Prejudice", a4 = "Jane Austen", i4 = "978-
0141439518";
    string t5 = "The Catcher in the Rye", a5 = "J.D. Salinger", i5 = "978-
0316769488";

    lib.addNewBook(t1, a1, i1);
    lib.addNewBook(t2, a2, i2);
    lib.addNewBook(t3, a3, i3);
    lib.addNewBook(t4, a4, i4);
    lib.addNewBook(t5, a5, i5);

    lib.displayDetails();
```

```
    delete[] dynamicBooks;
    return 0;
}
```

```
=== Library Books ===
Book 1:
  Title: The Great Gatsby
  Author: F. Scott Fitzgerald
  ISBN: 978-0743273565
Book 2:
  Title: 1984
  Author: George Orwell
  ISBN: 978-0451524935
Book 3:
  Title: To Kill a Mockingbird
  Author: Harper Lee
  ISBN: 978-0061120084
Book 4:
  Title: Pride and Prejudice
  Author: Jane Austen
  ISBN: 978-0141439518
Book 5:
  Title: The Catcher in the Rye
  Author: J.D. Salinger
  ISBN: 978-0316769488


...Program finished with exit code 0
```

Question 3

Create a class Account with attributes account number (const long), transaction ID, transaction type, and balance. Implement depositAmount, creditAmount, and displayDetails functions.

Code

```
#include <iostream>
#include <string>
using namespace std;

class Account {
private:
    const long accountNumber;
    long transactionId;
    string transactionType;
    double balance;
    static long nextTransactionId;

public:
    Account(long accNum, double initialBalance)
        : accountNumber(accNum), balance(initialBalance), transactionId(0),
transactionType("None") {}

    long depositAmount(const long &to, const long &from, const double &amount)
{
        transactionId = nextTransactionId++;
        transactionType = "Deposit";
        balance += amount;
        cout << "Deposited $" << amount << " to account " << to << endl;
        return transactionId;
    }

    long creditAmount(const long &to, const long &from, const double &amount) {
        transactionId = nextTransactionId++;
        transactionType = "Credit";
        if (balance >= amount) {
            balance -= amount;
            cout << "Credited $" << amount << " from account " << from << endl;
            return transactionId;
        }
        cout << "Insufficient balance!" << endl;
        return -1;
    }

    void displayDetails() const {
        cout << "=== Account Details ===" << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: $" << balance << endl;
        cout << "Last Transaction ID: " << transactionId << endl;
        cout << "Last Transaction Type: " << transactionType << endl;
    }
};
```

```
long Account::nextTransactionId = 1000;

int main() {
    Account acc1(1234567890, 5000.0);
    Account acc2(9876543210, 3000.0);

    acc1.depositAmount(acc1.accountNumber, 0, 1000.0);
    acc2.depositAmount(acc2.accountNumber, 0, 500.0);

    acc1.creditAmount(acc2.accountNumber, acc1.accountNumber, 200.0);

    cout << endl;
    acc1.displayDetails();
    cout << endl;
    acc2.displayDetails();

    return 0;
}
```

```
Deposited $1000 to account 1234567890
Deposited $500 to account 9876543210
Transferred $200 from account 1234567890 to account 9876543210

=== Account Details ===
Account Number: 1234567890
Balance: $5800
Last Transaction ID: 1002
Last Transaction Type: Transfer

=== Account Details ===
Account Number: 9876543210
Balance: $3700
Last Transaction ID: 1001
Last Transaction Type: Deposit


...Program finished with exit code 0
Press ENTER to exit console.
```

Question 4

Write a program to add data objects of two different classes using friend functions.

Code

```
#include <iostream>
using namespace std;

class ClassB;  // Forward declaration

class ClassA {
private:
    int valueA;
public:
    ClassA(int v) : valueA(v) {}
    friend void addObjects(ClassA &a, ClassB &b);
};

class ClassB {
private:
    int valueB;
public:
    ClassB(int v) : valueB(v) {}
    friend void addObjects(ClassA &a, ClassB &b);
};

void addObjects(ClassA &a, ClassB &b) {
    int sum = a.valueA + b.valueB;
    cout << "Sum of ClassA and ClassB values: " << sum << endl;
}

int main() {
    ClassA objA(50);
    ClassB objB(30);
    addObjects(objA, objB);
    return 0;
}
```

```
Sum of ClassA and ClassB values: 80


...Program finished with exit code 0
Press ENTER to exit console.
```

Question 5

Define a class named Complex with properties (real and imaginary) and methods: Parameterized constructor, copy constructor, display(), and sum() as friend function.

Code

```
#include <iostream>
using namespace std;

class Complex {
private:
    double real;
    double imaginary;
public:
    Complex(double r = 0, double i = 0) : real(r), imaginary(i) {}

    Complex(const Complex &other) {
        this->real = other.real;
        this->imaginary = other.imaginary;
    }

    void display() {
        cout << real << " + " << imaginary << "i" << endl;
    }

    friend Complex sum(Complex &c1, Complex &c2);
};

Complex sum(Complex &c1, Complex &c2) {
    Complex result;
    result.real = c1.real + c2.real;
    result.imaginary = c1.imaginary + c2.imaginary;
    return result;
}

int main() {
    Complex c1(3.5, 4.2);
    Complex c2(1.5, 2.8);

    cout << "Complex Number 1: ";
    c1.display();

    cout << "Complex Number 2: ";
    c2.display();

    Complex c3 = sum(c1, c2);
    cout << "Sum: ";
    c3.display();

    Complex c4(c1);  // Copy constructor
    cout << "Copy of c1: ";
    c4.display();
```

```
    return 0;
}
```

```
Complex Number 1: 3.5 + 4.2i
Complex Number 2: 1.5 + 2.8i
Sum: 5 + 7i
Copy of c1: 3.5 + 4.2i


...Program finished with exit code 0
Press ENTER to exit console.
```

Lab Assignment 7

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

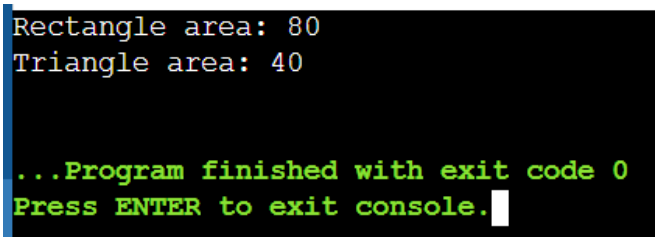
Kaushtubh Kumar

Roll No. 1024150191

Batch: 2O23

1. Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated

CODE	TESTCASES

`#include<iostream>`
`using namespace std;`
 	
`class polygon {`
`protected:`
`  int width, height;`
`public:`
`  void set_value(int w, int h) {`
`  width = w;`
`  height = h;`
`  }`
`};`
 	
`class rectangle : public polygon {`
`public:`
`  int calculate_area() {`
`  return width * height;`
`  }`
`};`
 	
`class triangle : public polygon {`
`public:`
`  int calculate_area() {`
`  return width * height / 2;`
`  }`
`};`
 	
`int main() {`
`  polygon *p;`
`  rectangle r;`
`  triangle t;`
 	
`  p = &r;`
`  p->set_value(10, 5);`
	

```
    cout << "Rectangle area: " <<
r.calculate_area() << endl;

    p = &t;
    p->set_value(10, 5);
    cout << "Triangle area: " <<
t.calculate_area() << endl;

    return 0;
}
}
```

2. Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

CODE

```
#include<iostream>
using namespace std;

class shape {
public:
    virtual void area() {}
    virtual void display() {}
};

class circle : public shape {
    float r;
public:
    circle(float x) { r = x; }
    void area() {
        cout << "Area of circle: " << 3.14 * r * r << endl;
    }
    void display() {
        cout << "Shape: Circle" << endl;
```

```
    }
};

class rectangle : public shape {
    float l, b;
public:
    rectangle(float x, float y) { l = x; b = y; }
    void area() {
        cout << "Area of rectangle: " << l * b << endl;
    }
    void display() {
        cout << "Shape: Rectangle" << endl;
    }
};

class triangle : public shape {
    float b, h;
public:
    triangle(float x, float y) { b = x; h = y; }
    void area() {
        cout << "Area of triangle: " << 0.5 * b * h <<
endl;
    }
    void display() {
        cout << "Shape: Triangle" << endl;
    }
};

int main() {
    shape *s;
    circle c(5);
    rectangle r(4, 6);
    triangle t(3, 7);

    s = &c; s->display(); s->area();
    s = &r; s->display(); s->area();
    s = &t; s->display(); s->area();

    return 0;
}
```

TESTCASE

```
Shape: Rectangle
Area of rectangle: 10
Shape: Triangle
Area of triangle: 17.5


...Program finished with exit code 0
Press ENTER to exit console.
```

3. Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

CODE	TESTCASES

`#include<iostream>`
`#include<math.h>`
`using namespace std;`
 	
`float area(float b, float h) {`
`    return 0.5 * b * h;`
`}`
 	
`float area(float a) {`
`    return (1.732 / 4) * a * a;`
`}`
 	
`float area(float a, float b, float c)`
`{`
`    float s = (a + b + c) / 2;`
`    return sqrt(s * (s - a) * (s - b)`
`* (s - c));`
`}`
 	
`int main() {`
` cout << "Right triangle: " <<`
`area(5, 6) << endl;`
`    cout << "Equilateral: " <<`
`area(9) << endl;`
	`Right triangle: 15`
	`Equilateral: 35.073`
	`Isosceles: 20.3332`
	
	`...Program finished with exit code 0`
	`Press ENTER to exit console.`

```
    cout << "Isosceles: " << area(6,
7, 8) << endl;

    return 0;
}
```

4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

CODE

```
#include<iostream>
using namespace std;

class Student {
public:
    virtual void show() = 0;
};

class Engineering : public Student {
public:
    void show() {
        cout << "Engineering student"
<< endl;
    }
};

class Medicine : public Student {
public:
    void show() {
        cout << "Medicine student" <<
endl;
    }
};

class Science : public Student {
public:
```

TESTCASES

```
Engineering student
Medicine student
Science student


...Program finished with exit code 0
Press ENTER to exit console.
```

```
    void show() {
        cout << "Science student" <<
endl;
    }
};

int main() {
    Student *s[3];
    Engineering e;
    Medicine m;
    Science sc;

    s[0] = &e;
    s[1] = &m;
    s[2] = &sc;

    for(int i = 0; i < 3; i++)
        s[i]->show();

    return 0;
}
```

5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.

CODE	**TESTCASES**

`#include<iostream>`
`using namespace std;`
 	
`class Time {`
`  int h, m, s;`
`public:`
`  Time() {}`
`  Time(int a, int b, int c) {`
`  h = a; m = b; s = c;`
`  }`
`  Time operator+(Time t) {`
	`46:20:49`
	
	`...Program finished with exit code 0`
	`Press ENTER to exit console.`

Time temp;
temp.s = s + t.s;
temp.m = m + t.m + temp.s / 60;
temp.s %= 60;
temp.h = h + t.h + temp.m / 60;
temp.m %= 60;
return temp;
}
void show() {
cout << h << ":" << m << ":" << s << endl;
}
};
 	
int main() {
Time t1(23, 45, 50), t2(22, 34, 59), t3;
t3 = t1 + t2;
t3.show();
return 0;
}	

6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings

CODE	**TESTCASES**

#include <iostream>
#include <string>
using namespace std;
 	
class STRING {
char s[100];
 	
public:
void input() {
	EQUAL STRINGS

```
        cin >> s;  // reads till
space
    }

    bool operator==(const STRING& t)
const {
        return strcmp(s, t.s) == 0;
    }

    STRING operator+(const STRING& t)
const {
        STRING temp;

        // Ensure no overflow
        if (strlen(s) + strlen(t.s)
>= 100) {
            cout << "String
overflow!" << endl;
            temp.s[0] = '\0';
            return temp;
        }

        strcpy(temp.s, s);
        strcat(temp.s, t.s);
        return temp;
    }

    void display() const {
        cout << s << endl;
    }
};

int main() {
    STRING s1, s2, s3;

    cout << "Enter first string: ";
    s1.input();

    cout << "Enter second string: ";
    s2.input();

    if (s1 == s2)
```

```
Enter first string: oops
Enter second string: oops
Equal
Concatenated string: oopsoops


...Program finished with exit code 0
Press ENTER to exit console.
```

UNEQUAL STRINGS

```
Enter first string: rashu
Enter second string: costubh
Not Equal
Concatenated string: rashucostubh


...Program finished with exit code 0
Press ENTER to exit console.
```

```
        cout << "Equal" << endl;
    else
        cout << "Not Equal" << endl;

    s3 = s1 + s2;

    cout << "Concatenated string: ";
    s3.display();

    return 0;
}
```

7. Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

CODE

```
#include<iostream>
using namespace std;

class Matrix {
    int a[10][10], r, c;
public:
    void input() {
        cout << "Enter rows and cols: ";
        cin >> r >> c;
        for(int i = 0; i < r; i++)
            for(int j = 0; j < c; j++)
                cin >> a[i][j];
    }
    void display() {
        for(int i = 0; i < r; i++) {
            for(int j = 0; j < c; j++)
                cout << a[i][j] << " ";
            cout << endl;
        }
    }
    friend Matrix operator*(Matrix, Matrix);
```

```
};

Matrix operator*(Matrix m1, Matrix m2) {
    Matrix m3;
    m3.r = m1.r;
    m3.c = m2.c;
    for(int i = 0; i < m3.r; i++) {
        for(int j = 0; j < m3.c; j++) {
            m3.a[i][j] = 0;
            for(int k = 0; k < m1.c; k++)
                m3.a[i][j] += m1.a[i][k] * m2.a[k][j];
        }
    }
    return m3;
}

int main() {
    Matrix m1, m2, m3;
    cout << "Matrix 1" << endl;
    m1.input();
    cout << "Matrix 2" << endl;
    m2.input();
    m3 = m1 * m2;
    cout << "Result" << endl;
    m3.display();
    return 0;
}
```

TESTCASE

```
Matrix 1:
Enter rows and columns: 2
2
Enter elements:
1
2
3
4
Matrix 2:
Enter rows and columns: 2
2
Enter elements:
4
5
6
7
Result:
16 19
36 43


...Program finished with exit code 0
Press ENTER to exit console.
```

8. Overload '[]' to check array index out of bounds problem at run time.

CODE	TESTCASES

`#include<iostream>`
`using namespace std;`
 	
`class Array {`
`  int a[10];`
`public:`
`  void input() {`
`  for(int i = 0; i < 10; i++)`
`cin >> a[i];`
`  }`
`  int operator[](int index) {`
`  if(index < 0 || index >=`
`10) {`
`  cout << "Index out of`
`bounds" << endl;`
`  return -1;`
`  }`
`  return a[index];`
`  }`
`};`
 	
`int main() {`
`  Array arr;`
`  arr.input();`
`  int i;`
`  cout << "Enter index: ";`
`  cin >> i;`
`  cout << arr[i] << endl;`
`  return 0;`
`}`
	VALID INDEX ELEMENT RETURNED
	`Enter array elements (MAX 5)34`
	`21`
	`12`
	`5`
	`55`
	`Enter index: 3`
	`5`
	
	`...Program finished with exit code 0`
	`Press ENTER to exit console.`
	
	INVALID INDEX ERROR THROWN
	`Enter array elements (MAX 5)4`
	`21`
	`33`
	`2`
	`1`
	`Enter index: 6`
	`Index out of bounds`
	`-1`
	
	`...Program finished with exit code 0`
	`Press ENTER to exit console.`

9. Overload '()' to input arbitrary number of input arguments for an object.

CODE	TESTCASES
`#include<iostream>` `using namespace std;`	

```
class Test {
public:
    void operator()(int a, int b,
int c) {
        cout << "Sum = " << a + b +
c << endl;
    }
};

int main() {
    Test t;
    t(10, 20, 30);
    return 0;
}
```

```
Sum = 60


...Program finished with exit code 0
Press ENTER to exit console.
```

10. Write a program in C++ to overload input operator (>>) and output operator (<<).

CODE	**TESTCASES**

`#include<iostream>`
`using namespace std;`
 	
`class Test {`
`    int x;`
`public:`
`    friend istream&`
`operator>>(istream &in, Test &t);`
`    friend ostream&`
`operator<<(ostream &out, Test &t);`
`};`
 	
`istream& operator>>(istream &in,`
`Test &t) {`
`    in >> t.x;`
`    return in;`
`}`
	`Enter value: 45`
	`Value: 45`
	
	`...Program finished with exit code 0`
	`Press ENTER to exit console.`

```
ostream& operator<<(ostream &out,
Test &t) {
    out << t.x;
    return out;
}

int main() {
    Test t;
    cout << "Enter value: ";
    cin >> t;
    cout << "Value: " << t << endl;
    return 0;
}
```

11. Write a program to convert basic data type (float) to user defined data type (object).

CODE	**TESTCASES**

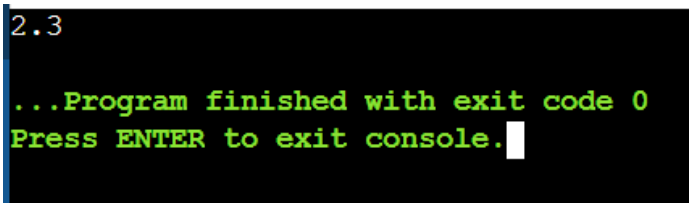
```
#include<iostream>
using namespace std;

class Test {
    float x;
public:
    Test(float f) {
        x = f;
    }
    void show() {
        cout << x;
    }
};

int main() {
    float f;
    cout << "Enter float: ";
    cin >> f;
    Test t = f;
    t.show();
```

```
Enter float: 45.2
45.2

...Program finished with exit code 0
Press ENTER to exit console.
```

`    return 0;` `}`	

12. Write a program to convert UDT to basic data type (float)

CODE	**TESTCASES**

`#include<iostream>`
`using namespace std;`
 	
`class Test {`
`    float x;`
`public:`
`    Test(float a) {`
`        x = a;`
`    }`
`    operator float() {`
`        return x;`
`    }`
`};`
 	
`int main() {`
`    Test t(5.5);`
`    float f = t;`
`    cout << f;`
`    return 0;`
`}`
	

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system

CODE	**TESTCASES**

```
#include<iostream>
#include<math.h>
using namespace std;

class Cartesian;

class Polar {
    float r, theta;
public:
    Polar(float a, float b) {
        r = a;
        theta = b;
    }
    operator Cartesian();
};

class Cartesian {
    float x, y;
public:
    Cartesian() {}
    Cartesian(float a, float b) {
        x = a;
        y = b;
    }
    void show() {
        cout << "x = " << x << " y
= " << y << endl;
    }
};

Polar::operator Cartesian() {
    float x = r * cos(theta);
    float y = r * sin(theta);
    return Cartesian(x, y);
}

int main() {
    Polar p(2,3);
    Cartesian c = p;
    c.show();
    return 0;
}
```

```
x = -3.95997 y = 0.56448


...Program finished with exit code 0
Press ENTER to exit console.
```


CODE	**TESTCASES**

Lab Assignment 8
Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Kaushtubh Kumar

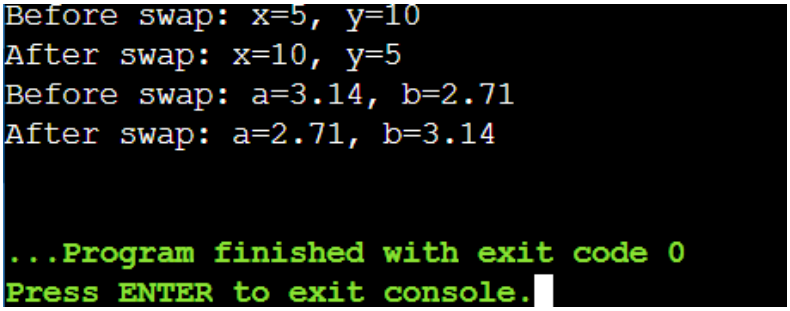
Roll No. 1024150191

Batch: 2O23

Function Templates

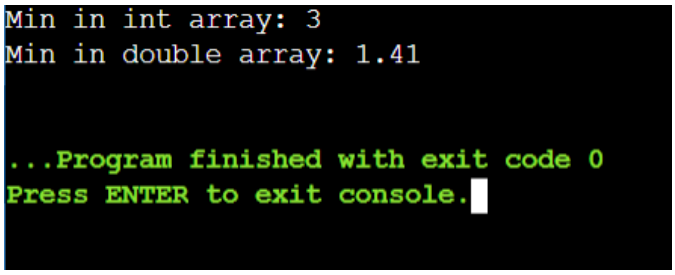
1. Write a function template to swap two variables of any data type.

Code	Testcase

`#include <iostream>`
`using namespace std;`
 	
`template <typename T>`
`void swapValues(T &a, T &b) {`
`    T temp = a;`
`    a = b;`
`    b = temp;`
`}`
 	
`int main() {`
`    int x = 5, y = 10;`
`    cout << "Before swap: x=" << x << ", y=" << y << endl;`
`    swapValues(x, y);`
`    cout << "After swap: x=" << x << ", y=" << y << endl;`
 	
`    double a = 3.14, b = 2.71;`
`    cout << "Before swap: a=" << a << ", b=" << b << endl;`
`    swapValues(a, b);`
`    cout << "After swap: a=" << a << ", b=" << b << endl;`
 	
`    return 0;`
`}`
	

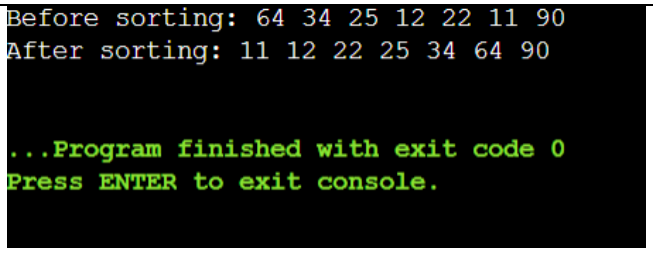
2. Write a function template to find the minimum element in an array.

Code	Testcase

`#include <iostream>`
`using namespace std;`
 	
`template <typename T>`
`T findMin(T arr[], int size) {`
`    T min = arr[0];`
`    for (int i = 1; i < size; i++) {`
`        if (arr[i] < min) {`
`            min = arr[i];`
`        }`
`    }`
`    return min;`
`}`
 	
`int main() {`
`    int intArr[] = {45, 12, 78, 3, 89, 25};`
`    cout << "Min in int array: " << findMin(intArr, 6) << endl;`
 	
`    double doubleArr[] = {3.14, 2.71, 1.41, 5.67};`
`    cout << "Min in double array: " << findMin(doubleArr, 4) << endl;`
 	
`    return 0;`
`}`
	

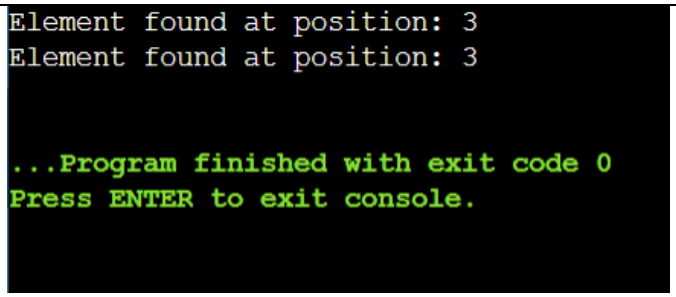
3. Write a function template to sort an array using bubble sort.

Code	Testcase

<pre>#include <iostream>
using namespace std;
 	
template <typename T>
void bubbleSort(T arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
 	
template <typename T>
void display(T arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}
 	
int main() {
    int intArr[] = {64, 34, 25, 12, 22, 11, 90};
    cout << "Before sorting: ";
    display(intArr, 7);
    bubbleSort(intArr, 7);
    cout << "After sorting: ";
    display(intArr, 7);
 	
    return 0;
}</pre>
	

4. Write a function template to perform linear search.

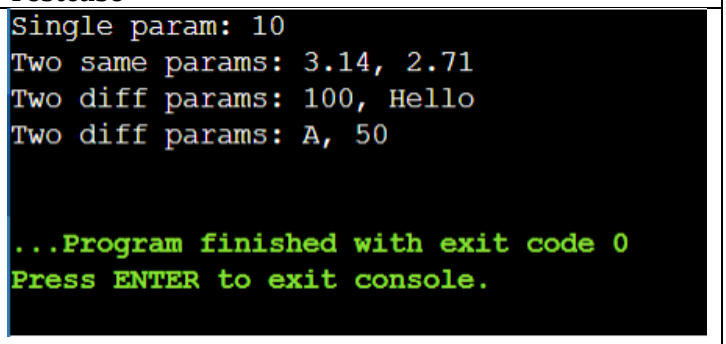
Code	Testcase

`#include <iostream>`
`using namespace std;`
 	
`template <typename T>`
`int linearSearch(T arr[], int size, T key) {`
`    for (int i = 0; i < size; i++) {`
`        if (arr[i] == key) {`
`            return i;`
`        }`
`    }`
`    return -1;`
`}`
 	
`int main() {`
`    int intArr[] = {10, 25, 35, 45, 55};`
`    int pos = linearSearch(intArr, 5, 35);`
`    if (pos != -1)`
`        cout << "Element found at position: " << pos + 1 << endl;`
`    else`
`        cout << "Element not found" << endl;`
 	
`    char charArr[] = {'A', 'B', 'C', 'D'};`
`    pos = linearSearch(charArr, 4, 'C');`
`    if (pos != -1)`
`        cout << "Element found at position: " << pos + 1 << endl;`
 	
`    return 0;`
`}`
	

5. Design overloaded versions of a function template process() such that:

i) Accepts a single parameter
ii) Accepts two parameters of the same type
iii) Accepts two parameters of different types

Code	Testcase

`#include <iostream>`
`using namespace std;`
 	
`// Single parameter`
`template <typename T>`
`void process(T a) {`
`    cout << "Single param: " << a << endl;`
`}`
 	
`// Two parameters of same type`
`template <typename T>`
`void process(T a, T b) {`
`    cout << "Two same params: " << a << ", " << b <<`
`endl;`
`}`
 	
`// Two parameters of different types`
`template <typename T, typename U>`
`void process(T a, U b) {`
`    cout << "Two diff params: " << a << ", " << b <<`
`endl;`
`}`
 	
`int main() {`
`    process(10);`
`    process(3.14, 2.71);`
`    process(100, "Hello");`
`    process('A', 50);`
`    return 0;`
`}`
	

Class Templates

6. Develop a class template for stack with push and pop operations.

Code

```
#include <iostream>
using namespace std;

template <typename T>
class Stack {
private:
    T* arr;
    int top;
    int capacity;

public:
    Stack(int size = 10) {
        arr = new T[size];
        top = -1;
        capacity = size;
    }

    ~Stack() {
        delete[] arr;
    }

    void push(T value) {
        if (top >= capacity - 1) {
            cout << "Stack Overflow" << endl;
            return;
        }
        arr[++top] = value;
        cout << "Pushed: " << value << endl;
    }

    T pop() {
        if (top < 0) {
            cout << "Stack Underflow" << endl;
            return T();
        }
        return arr[top--];
    }

    void display() {
        for (int i = top; i >= 0; i--)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Stack<int> intStack;
    intStack.push(10);
    intStack.push(20);
    intStack.push(30);
    cout << "Stack: ";
    intStack.display();
    cout << "Popped: " << intStack.pop() << endl;

    return 0;
}
```

Testcase

```
Pushed: 10
Pushed: 20
Pushed: 30
Stack: 30 20 10
Popped: 30


...Program finished with exit code 0
Press ENTER to exit console.
```

7. Develop a class template for queue with enqueue and dequeue operations.

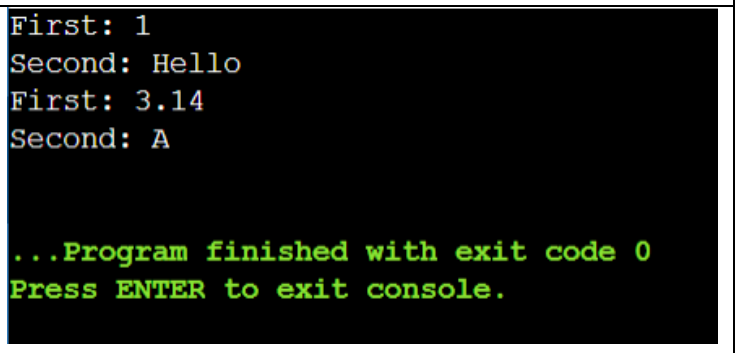
Code	Testcase

```
#include <iostream>
using namespace std;

template <typename T>
class Queue {
private:
    T* arr;
    int front, rear;
    int capacity;

public:
    Queue(int size = 10) {
        arr = new T[size];
        front = 0;
        rear = -1;
        capacity = size;
    }

    ~Queue() {
        delete[] arr;
    }

    void enqueue(T value) {
        if (rear >= capacity - 1) {
            cout << "Queue Overflow" << endl;
            return;
        }
        arr[++rear] = value;
        cout << "Enqueued: " << value << endl;
    }

    T dequeue() {
        if (front > rear) {
            cout << "Queue Underflow" << endl;
            return T();
        }
        return arr[front++];
    }

    void display() {
        for (int i = front; i <= rear; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Queue<int> q;
    q.enqueue(100);
    q.enqueue(200);
    q.enqueue(300);
    cout << "Queue: ";
    q.display();
    cout << "Dequeued: " << q.dequeue() << endl;

    return 0;
}
```

```
Enqueued: 100
Enqueued: 200
Enqueued: 300
Queue: 100 200 300
Dequeued: 100


...Program finished with exit code 0
Press ENTER to exit console.
```

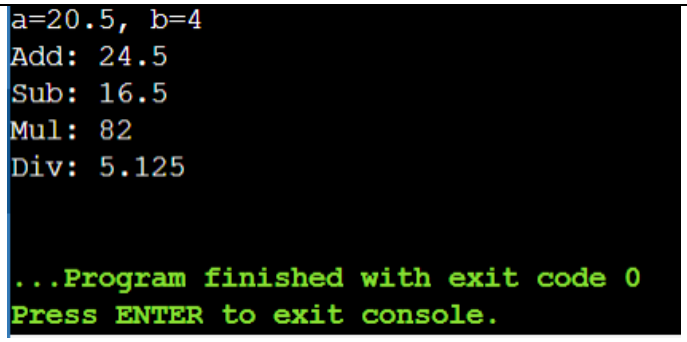
8. Create a class template to store and display a pair of values.

Code	Testcase

`#include <iostream>`
`using namespace std;`
 	
`template <typename T1, typename T2>`
`class Pair {`
`private:`
`    T1 first;`
`    T2 second;`
 	
`public:`
`    Pair(T1 f, T2 s) {`
`        first = f;`
`        second = s;`
`    }`
 	
`    void display() {`
`        cout << "First: " << first << endl;`
`        cout << "Second: " << second << endl;`
`    }`
 	
`    T1 getFirst() { return first; }`
`    T2 getSecond() { return second; }`
`};`
 	
`int main() {`
`    Pair<int, string> p1(1, "Hello");`
`    p1.display();`
 	
`    Pair<double, char> p2(3.14, 'A');`
`    p2.display();`
 	
`    return 0;`
`}`
	

9. Create a class template for basic arithmetic operations.

Code	Testcase

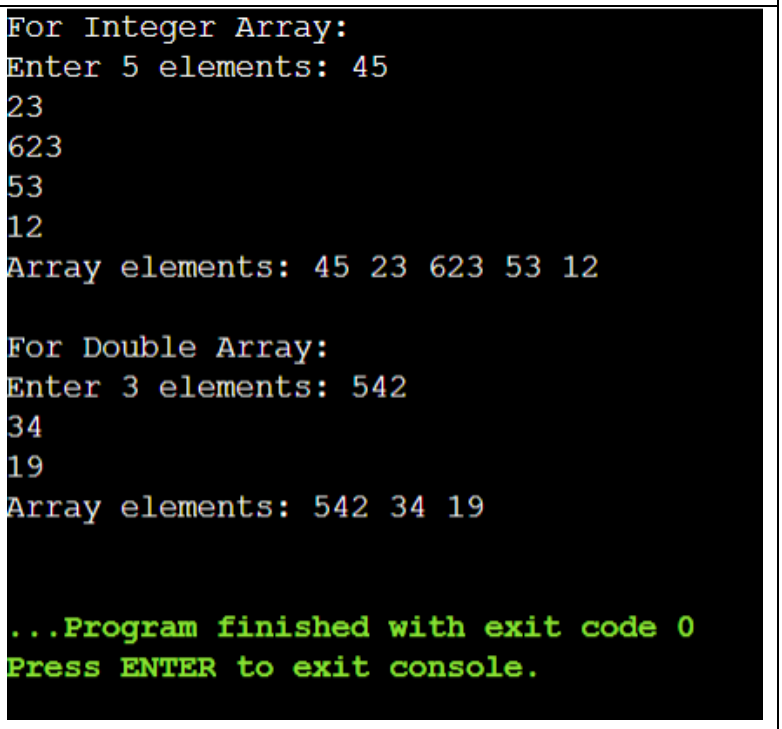
#include <iostream>
using namespace std;
 	
template <typename T>
class Arithmetic {
private:
    T a, b;
 	
public:
    Arithmetic(T x, T y) {
        a = x;
        b = y;
    }
 	
    T add() { return a + b; }
    T subtract() { return a - b; }
    T multiply() { return a * b; }
    T divide() {
        if (b != 0)
            return a / b;
        else {
            cout << "Division by zero!" << endl;
            return T();
        }
    }
 	
    void display() {
        cout << "a=" << a << ", b=" << b << endl;
        cout << "Add: " << add() << endl;
        cout << "Sub: " << subtract() << endl;
        cout << "Mul: " << multiply() << endl;
        cout << "Div: " << divide() << endl;
    }
};
 	
int main() {
    Arithmetic<int> calc1(10, 5);
    calc1.display();
 	
    Arithmetic<double> calc2(20.5, 4.0);
    calc2.display();
 	
    return 0;
}
	

10. Create a class template to input and display array elements.

Code

```
#include <iostream>
using namespace std;

template <typename T>
class Array {
private:
    T* arr;
    int size;

public:
    Array(int s) {
        size = s;
        arr = new T[size];
    }

    ~Array() {
        delete[] arr;
    }

    void input() {
        cout << "Enter " << size << " elements: ";
        for (int i = 0; i < size; i++)
            cin >> arr[i];
    }

    void display() {
        cout << "Array elements: ";
        for (int i = 0; i < size; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Array<int> intArr(5);
    cout << "For Integer Array:" << endl;
    intArr.input();
    intArr.display();

    Array<double> doubleArr(3);
    cout << "\nFor Double Array:" << endl;
    doubleArr.input();
    doubleArr.display();

    return 0;
}
```

Testcase



Lab Assignment 9

OOPS

Branch

B.E., 2nd Year – ENC

Submitted By:

Kaushtubh Kumar

Roll No. 1024150191

Batch: 2O23

Assignment-9

File Management

Question 1

Write a C++ program to write number 1 to 200 in a data file NUM.TXT.

Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("NUM.TXT");
    if (!outFile) {
        cout << "Error opening file!" << endl;
        return 1;
    }
    for (int i = 1; i <= 200; i++) {
        outFile << i << endl;
    }
    outFile.close();
    cout << "Numbers 1 to 200 written to NUM.TXT successfully!" << endl;
    return 0;
}
```

Output / Test Case Result:

```
Numbers 1 to 200 written to NUM.TXT successfully!

[NUM.TXT file created with contents:]
1
2
3
...
198
199
200
```

Question 2

Write a user-defined function in C++ to read the content from a text file NOTES.TXT, count and display the number of alphabets present in it.

Code:

```
#include <iostream>
#include <fstream>
#include <cctype>
using namespace std;

void countAlphabets() {
    ifstream inFile("NOTES.TXT");
    if (!inFile) {
        cout << "Error opening file!" << endl;
        return;
    }
    char ch;
    int count = 0;
    while (inFile.get(ch)) {
        if (isalpha(ch)) {
            count++;
```

```
        }
    }
    inFile.close();
    cout << "Number of alphabets in NOTES.TXT: " << count << endl;
}

int main() {
    countAlphabets();
    return 0;
}
```

Output / Test Case Result:

Assumption: NOTES.TXT contains "Hello World! This is C++." (22 alphabetic characters)

```
Number of alphabets in NOTES.TXT: 22
```

Question 3

Write a program to copy the contents of one file to another.

Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream sourceFile("source.txt");
    ofstream destFile("destination.txt");

    if (!sourceFile) {
        cout << "Error opening source file!" << endl;
        return 1;
    }
    if (!destFile) {
        cout << "Error creating destination file!" << endl;
        return 1;
    }

    char ch;
    while (sourceFile.get(ch)) {
        destFile.put(ch);
    }

    sourceFile.close();
    destFile.close();
    cout << "File copied successfully!" << endl;
    return 0;
}
```

Output / Test Case Result:

Assumption: source.txt contains "Hello, this is a test file."

```
File copied successfully!

[destination.txt now contains:]
Hello, this is a test file.
```

Question 4

Write a program to implement I/O operations on characters. I/O operations include inputting a string, calculating the length of the string, storing the string in a file and fetching the stored characters from it.

Code:

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main() {
    char str[100];
    int length;

    // Input string
    cout << "Enter a string: ";
    cin.getline(str, 100);

    // Calculate length
    length = strlen(str);
    cout << "Length of string: " << length << endl;

    // Store in file
    ofstream outFile("string.txt");
    if (!outFile) {
        cout << "Error opening file!" << endl;
        return 1;
    }
    outFile << str;
    outFile.close();
    cout << "String stored in file!" << endl;

    // Fetch from file
    ifstream inFile("string.txt");
    if (!inFile) {
        cout << "Error opening file!" << endl;
        return 1;
    }
    char ch;
    cout << "Content from file: ";
    while (inFile.get(ch)) {
        cout << ch;
    }
    cout << endl;
    inFile.close();
    return 0;
}
```

Output / Test Case Result:

Input: "Hello C++"

```
Enter a string: Hello C++
Length of string: 9
String stored in file!
Content from file: Hello C++
```

Question 5

Write a program to do the followings:

(a) Create a file A-Z and read 10th character using seekg().

(b) Overwrite 5th number using seekp().

(c) Find file size using tellg().

(d) Read last character using seekg().

(e) Create a file data.txt with multiple lines. Read the file and:

- Move the pointer to the 10th byte using seekg()
- Display the current position using tellg().
- Read and display the remaining content.

Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // (a) Create file A-Z
    ofstream outFile("A-Z.txt");
    for (char ch = 'A'; ch <= 'Z'; ch++) {
        outFile.put(ch);
    }
    outFile.close();

    // Read 10th character using seekg()
    ifstream inFile("A-Z.txt");
    inFile.seekg(9); // 10th character (0-indexed)
    char ch;
    inFile.get(ch);
    cout << "10th character: " << ch << endl;
    inFile.close();

    // (b) Overwrite 5th position using seekp()
    ofstream outFile2("numbers.txt");
    for (int i = 1; i <= 10; i++) {
        outFile2 << i << " ";
    }
    outFile2.close();
    fstream file("numbers.txt", ios::in | ios::out);
    file.seekp(0); // Move to beginning
    file << "99"; // Overwrite with new value
    file.close();
    cout << "5th position overwritten with 99" << endl;

    // (c) Find file size using tellg()
    ifstream file3("A-Z.txt");
    file3.seekg(0, ios::end);
    int size = file3.tellg();
    cout << "File size: " << size << " bytes" << endl;
    file3.close();

    // (d) Read last character using seekg()
    ifstream file4("A-Z.txt");
    file4.seekg(-1, ios::end);
    file4.get(ch);
    cout << "Last character: " << ch << endl;
    file4.close();

    // (e) Create data.txt with multiple lines
    ofstream dataFile("data.txt");
    dataFile << "Line 1: Hello World" << endl;
    dataFile << "Line 2: C++ Programming" << endl;
    dataFile << "Line 3: File Handling" << endl;
    dataFile.close();

    ifstream readFile("data.txt");
    readFile.seekg(10); // Move to 10th byte
    int pos = readFile.tellg();
```

```
    cout << "Current position: " << pos << endl;
    cout << "Remaining content:" << endl;
    char c;
    while (readFile.get(c)) {
        cout << c;
    }
    cout << endl;
    readFile.close();

    return 0;
}
```

Output / Test Case Result:

```
10th character: J
5th position overwritten with 99
File size: 26 bytes
Last character: Z
Current position: 10
Remaining content:
Hello World
Line 2: C++ Programming
Line 3: File Handling
```

Question 6

Write a program to do the followings:

- Create a file with text "Hello World"
- After writing each character, display the current position of the put pointer using tellp().
- Replace "World" with "C++" using random access.

Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Create file and write "Hello World"
    fstream file("hello.txt", ios::out | ios::in | ios::trunc);
    if (!file) {
        // Create file first
        ofstream createFile("hello.txt");
        createFile.close();
        file.open("hello.txt", ios::out | ios::in);
    }

    string text = "Hello World";
    cout << "Writing characters:" << endl;
    for (int i = 0; i < text.length(); i++) {
        file.put(text[i]);
        int pos = file.tellp();
        cout << "Written '" << text[i] << "' at position: " << pos << endl;
    }

    // Replace "World" with "C++" using random access
    // "World" starts at position 6
    file.seekp(6);
    file << "C++ "; // Overwrite with C++ and spaces

    file.seekg(0);
    char ch;
```

```
    cout << "\nUpdated content: ";
    while (file.get(ch)) {
        cout << ch;
    }
    cout << endl;
    file.close();
    return 0;
}
```

Output / Test Case Result:

```
Writing characters:
Written 'H' at position: 1
Written 'e' at position: 2
Written 'l' at position: 3
Written 'l' at position: 4
Written 'o' at position: 5
Written ' ' at position: 6
Written 'W' at position: 7
Written 'o' at position: 8
Written 'r' at position: 9
Written 'l' at position: 10
Written 'd' at position: 11

Updated content: Hello C++
```